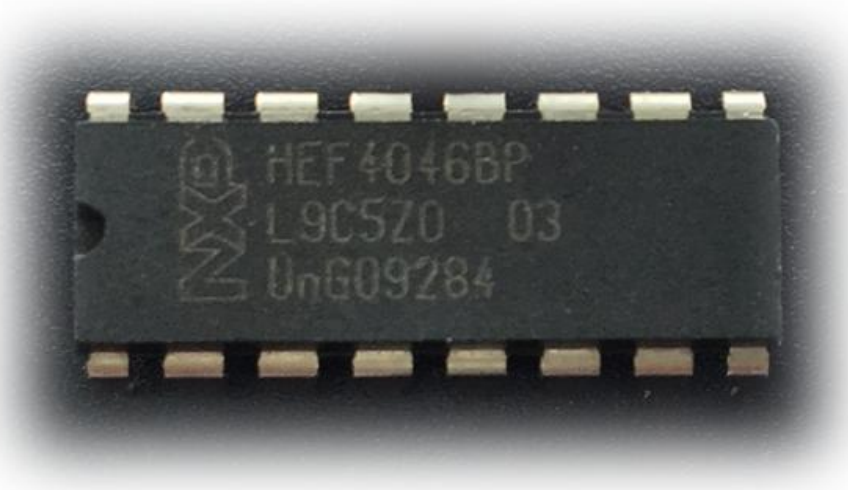
Phase-Locked Loop Integrated Circuits

PART II PRACTICALS E1 & E2 CLASS MANUAL

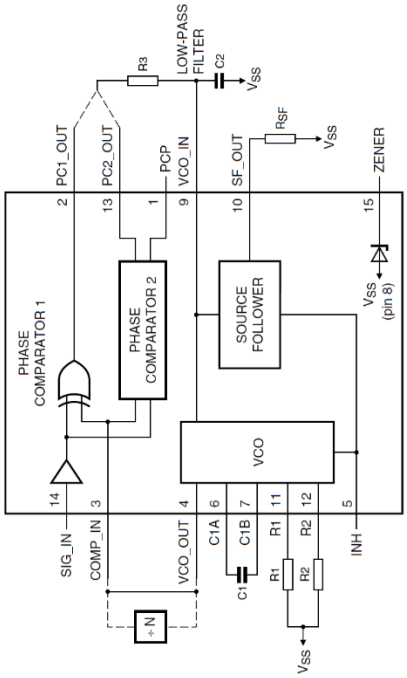
CAVENDISH LABORATORY, UNIVERSITY OF CAMBRIDGE

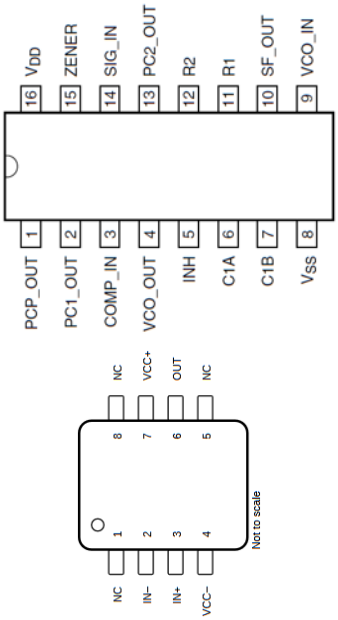
Head of Class: Andy Irvine (aci20@cam.ac.uk)

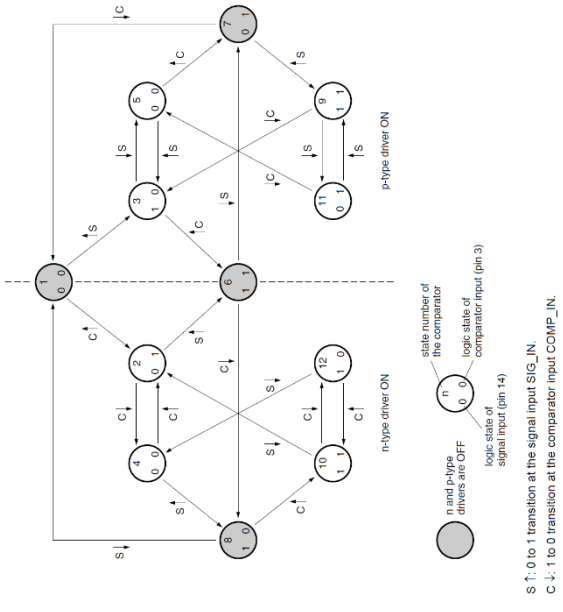
Original experiment and manuscript J. R. A. Cleaver, 2002
Revised and updated A. C. Irvine, 2019, 2025 and 2026
Python upgrade and further development M. Kals, S. Wolanski and P. Cicuta, 2023

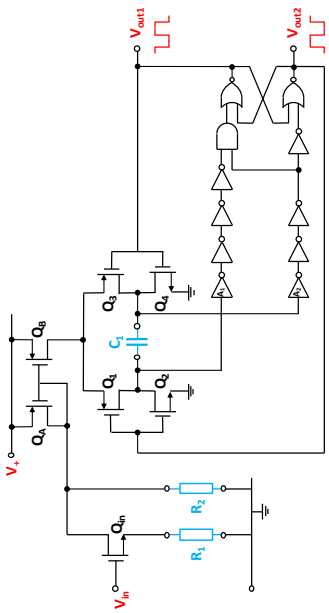


Selected useful images for easy reference









Contents

Overview 5
1 Background 5
1.1 The Experimental System 5
1.2 Methodology and Goals – Important! 5
1.3 Development and Retention of Code 6
1.4 Conducting the Experiment and Writing Up 6
1.5 Artificial Intelligence (AI) 6
Computer-Based Control and Measurement 7
2 Overview of Measurement Equipment with Examples 7
2.1 Software and Hardware 7
2.2 PicoScope 7
2.3 Arduino UNO 8
2.4 Python 10
Background to the Experiment 16
3 Introduction to Phase-Locked Loops 16
3.1 The Experiment 16
4 Phase-Locked Loop System Operation 17
4.1 Feedback and Stability 17
4.2 Effects of Filter Circuits on Stability 17
4.3 Tracking and Capture 18
5 The CMOS Integrated Circuit Type 4046 Phase-Locked Loop 19
5.1 General Outline 19
5.2 Phase Comparator Type 1: EXCLUSIVE-OR Circuit 20
5.3 Phase Comparator Type 2: Edge-Triggered Circuit 20
5.4 Voltage-Controlled Oscillator 22
5.5 Quiescent Conditions and Signal Capture 23
5.6 Noise and Interference 24
The Experiment 25
6 Experimental Characterisation of the Phase-Locked Loop Integrated Circuit 25
6.1 Useful Circuits 25
6.2 Phase Comparator Characterisation - Phase-Shift Supply 26
6.3 Phase-Comparator Characterisation - Type 1 Comparator 27
6.4 Phase-Comparator Characterisation - Type 2 Comparator 27
6.5 Voltage-Controlled Oscillator Characterisation 28
6.6 Basic Closed-Loop Operation 28

7 Applications and Investigations 29
7.1 Frequency Multiplier 29
7.2 Transient Response and Frequency Modulation 29
7.3 Clock Regeneration 30
7.4 Input MOSFET Characterisation 32
7.5 Other Relevant Applications 32
References 33
Selected Further Reading 33
Appendices 34
A1 Experimental Apparatus 34
A1.1 The Prototyping Board 34
A1.2 Using your own Laptop 34
A1.3 Programming the UNO – Advanced Notes 35
A2 The Phase-Locked Loop as a Control System 39
A2.1 Linear, Small-Signal Analysis for Stability 39
A2.2 Effects of Filter Circuits on Stability 40
A2.3 Gain, Margin, Phase Margin and Acceptable System Performance 42
A2.4 Bode Diagrams 44
A3 Integrated-Circuit Connections 46
A3.1 HEF4046B and 74HC4046 Phase-Locked Loops 46
A3.2 µA741 Operational Amplifier 47
A3.3 TL071 Operational Amplifier 47
A3.4 LM393A Dual Comparator 48
A3.5 HEF4015B Dual 4-Bit Static Shift Register 48
A3.6 HEF4020B and CD4020BE 14-Stage Binary Counters 49
A3.7 HEF4081 B Quadruple 2-Input AND Gate 49
A4 Operational Amplifier Circuits 50
A5 Physical Layout of the HEF4046B 51
A5.1 Introduction 51
A5.2 Integrated-Circuit Layout and Operation 51
A5.3 Example 1: Four-Input NOR Gate 53
A5.4 Process Flow 53
A5.5 Example 2: Large Channel-Width Transistor 56

Overview

1 Background

1.1 The Experimental System

This project is essentially an investigation of an implementation of the phase-locked loop, a mixed digital and analogue circuit based on the 4046 family of sixteen-pin chip packages; its operation is subtle and powerful, and it has many uses. In addition, much emphasis will be placed on getting a taste of data acquisition by computer, involving the logging of relatively large data sets in an automated fashion by means of custom-built and adapted programs.

The HEF4046B phase-locked loop chip forms part of a feedback circuit which allows it to 'lock on' to a digital waveform, producing its own output which has a well-controlled frequency and phase relationship to the test signal. It consists of two principal parts; a phase comparator which produces an 'error' signal which depends on the phase difference between two inputs, and a voltage-controlled oscillator which, as the name suggests, produces a digital output of frequency dependent on the voltage applied to its input. The frequency range attainable depends on external electronic components chosen by the user.

The experimenter will investigate the performances of the phase comparators and the voltage-controlled oscillator independently, before constructing a feedback circuit, in which complex feedback behaviours will be investigated.

Data acquisition will involve elements of computerised automation. The phase comparators will be studied using an Arduino UNO microcontroller, which the experimenter will program to use as a multi-channel pulse generator allowing a fixed phase difference between channels. The programming language Python will be used to control and read out from a PicoScope digital oscilloscope. Note that in order not to disadvantage those without a strong computer science background, **no great amount of programming experience is necessary, nor should such experience confer any substantial advantage**. The minimum amount of Python required in your studies so far should be plenty, and even without that, common sense and/or a good Demonstrator should get you through fairly painlessly.

1.2 Methodology and Goals – Important!

Some things to bear in mind:

- This project has been adapted from a more hands-on electronics-based experiment; as such, the timescales for completing the various elements are different now. It is NOT expected (nor was it in the past) that all tasks should be completed, and it is FAR more important that the appropriate experimental techniques, precautions and analysis have been applied. It is reasonably important, however, to have investigated (and understood) the phase comparators and voltage-controlled oscillator (Sections 6.2 to 6.5), and to carry out a reasonably thorough investigation of the basic closed loop (Section 6.6). The example applications of Section 7 will be helpful to your final mark if your investigation does them justice, but there are no prizes for ticking the most boxes.
- Discovering and proving the characteristics of the devices under test are the key here, but it is an excellent opportunity to make use of your laptop to acquire a large set of data; though a manually-plotted graph with half a dozen points may prove a hypothesis, a curve (or family of curves) with hundreds or thousands of data points may be more convincing PROVIDED you do not forget the scientific justification, errors and so on. This experiment allows you to explore the power of automation, and it is expected that you will make use of it in some way.

- A physics experiment is supposed to reveal the underlying principles of the system of interest and is quantitative as well as qualitative.
- This project is far from perfect - intentionally - and it is crucial that you deal with and record imperfections in the appropriate manner.

1.3 Development and Retention of Code

As you will already know, you are expected to submit a formal write-up of the experiment for assessment. Code does not need to be submitted, unless you choose to do so, but do please keep a copy in case it is needed to help with the marking. You will NOT need to keep any of your code from Section 2, which is intended as a practice and learning exercise. Programs you create or modify for use in the experimental phase, Sections 6 and 7, should be kept safe.

You will NOT be assessed on the quality or beauty of your coding, only on your ideas of what measurements to take. The science of the experimental circuit is all that matters. If it is requested, code may serve the purpose of helping the Head of Class to figure out where any problems arose. Requiring you to keep it will also, hopefully, discourage plagiarism (remember that passing your code to a colleague might disadvantage you once marks are moderated!).

If you get time, please remove code, data and so on from the lab laptop – but only once you're certain that you've made reliable backups!

1.4 Conducting the Experiment and Writing Up

You will be familiar with the excellent document published by the Teaching Office with the title "Keeping Laboratory Notes and Writing Formal Reports". It contains excellent advice, and is easily found on the TIS. However, do not assume it must be obeyed slavishly; projects have their own quirks, and there is no universal "correct" structure.

1.5 Artificial Intelligence (AI)

This is an experiment, and associated write-up, with the purpose of both acquiring and deploying skills, and the use of AI to do significant areas of work for you is not an appropriate way of achieving those aims. At all times you are required to obey any rules published by the Teaching Office and/or the Senior Examiner. My guidance is subservient to that. If, and only if, it is possible given those constraints to use AI – carefully - for trivial assistance (particularly in saving time), I am happy to permit it. You should NOT use AI as your source of facts or literature unless the information is checked thoroughly, and if you do, you should be aware that the process can and does introduce bias, and you must take care to remedy this. You certainly must not ask AI to write your report (or substantial parts of it) for you (we are likely to check). On the other hand, as this is not a computing project, having AI auto-complete your code, or even design small routines, is fine by me, since your intellect will still be guiding the process. For all aspects of any of this, ask if unsure.

Computer-Based Control and Measurement

2 Overview of Measurement Equipment with Examples

2.1 Software and Hardware

In the PLL Lab you are provided with a laptop, a PicoScope digital oscilloscope and an Arduino Uno microcontroller. You will see on the desktop of the laptop a folder named "E1 E2 PLL files master", which contains the electronic form of this manual, some chip data sheets and a few basic test programs for the control of the PicoScope and Arduino. All of the necessary software should have been installed on the laptop such that basic test programs provided work "out of the box" (ask/contact your Demonstrator urgently if not). This folder is located in the desktop folder of the "Public" user, and you will not have permissions to modify it. You should therefore make a copy in an appropriate location for editing.

This being a physics department, it is likely that you may be keen to use your own laptop. This is fine, but you should do so at your own risk and without expecting assistance. At the moment, only Windows 11 machines have been tried, and we can give no advice for other operating systems – I can only assume the setup is similar for them. The PicoScope and Arduino software packages can be downloaded from the respective company websites and will almost inevitably work very well straight away. Making Python drive the PicoScope is a little trickier, and there are some helpful tips for doing so on your own machine in Appendix A1.2. You should also think about whether your USB ports (and anything else) are compatible with the apparatus on your bench – we will not usually provide additional cables or hardware.

2.2 PicoScope

PicoScope is a staple of the IA and IB experimental classes, and you will be thoroughly familiar with it. Consequently, this manual will not go into detail about it (there is a User Guide available from the on-screen 'Help' menu). The model used in this experiment is the PicoScope 2406B (pico Technology, n.d.), which has four voltage input channels and an arbitrary waveform generator (AWG), which does (broadly) what it says it does. This project provides support for using the AWG to produce some basic waveforms (including square, sinusoidal and ramp). If you wish to do something genuinely "arbitrary" with the AWG, you're on your own. The PicoScope has a bandwidth of 50 MHz and an eight-bit sample rate of 1 GS/s. Its 32 MS onboard memory is suitable for recording some very large data sets, should you be able to handle them. In the usual way, the PicoScope should be plugged into your laptop using the standard USB type-A-to-type-B cable provided. Occasionally it needs disconnecting and re-connecting if the laptop can't recognise it. Another troubleshooting tip is to remember that PicoScope may only be accessed by one program at a time, and it may not be detected if already in use elsewhere.

The software will be familiar to you (Figure 1); this lab uses the PicoScope 7 T&M version. Use the PicoScope with this proprietary software whenever you see fit. If you wish to use the 'x10' probe feature to obtain higher input impedance, remember to calibrate the scope probes by adjusting the trimmer to avoid over/undershoot (recall IB labs!), and remember also to use the 'x10' software setting to compensate for attenuation. In many circumstances a standard 'x1' setting will be less bother, but do think about why that is or is not the case.

However, you are likely to find that you will take your "proper" data using Python, of which more later.

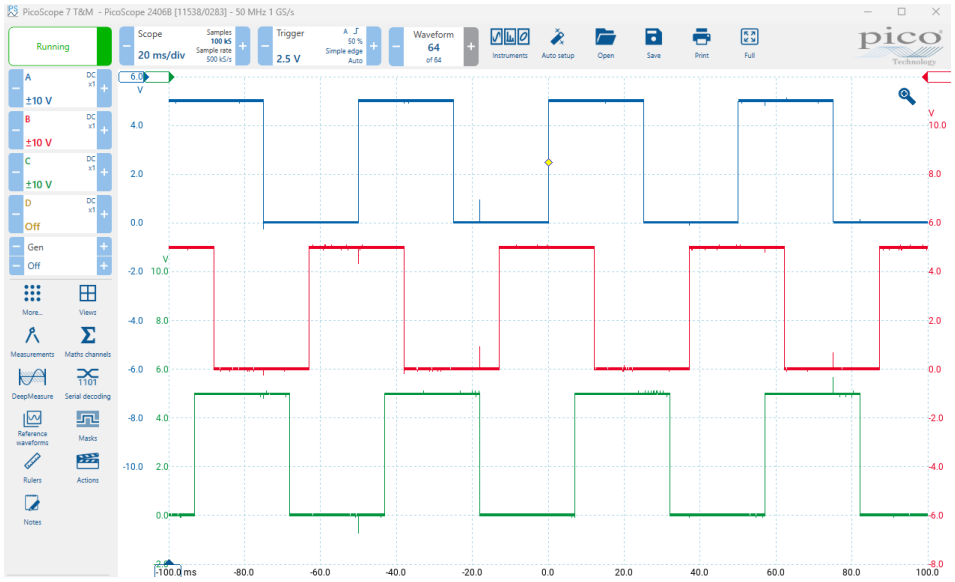


Figure 1 - PicoScope 7 T&M software GUI.

2.3 Arduino UNO

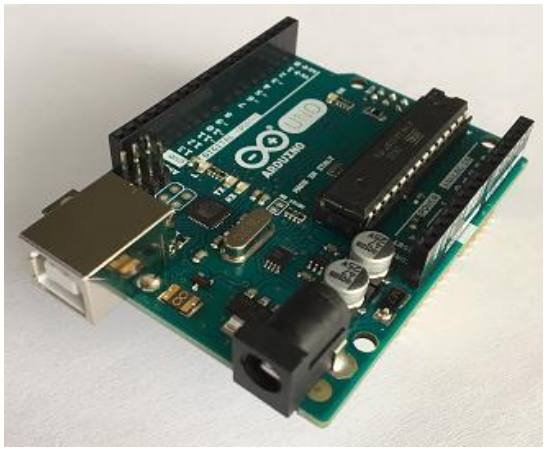


Figure 2 - Arduino UNO.

Arduino is a series of small, inexpensive microcontrollers based on open-source technology, and available in a range of specifications. It has gained a strong following amongst hobbyists and is a powerful tool capable of sophisticated applications. It is a handy demonstrator for the principles of experimental control and measurement, whilst retaining enough drawbacks to challenge the experimenter's technique. The *UNO* model (Figure 2) will be used in this project, incorporating a 16 MHz ATmega328P processor. In this experiment it is perfectly adequate (and convenient) to power it using only the UNO's A-to-B USB connector lead, through which the unit may also be programmed. For those interested, full specifications may be found online for the Arduino UNO (Arduino AG, n.d.) and the ATmega328P (Atmel Corporation).

Again, remember that this section is not intended for submission within your write-up - just go through it to familiarise yourself with the Arduino.

2.3.1 Input/Output Pins

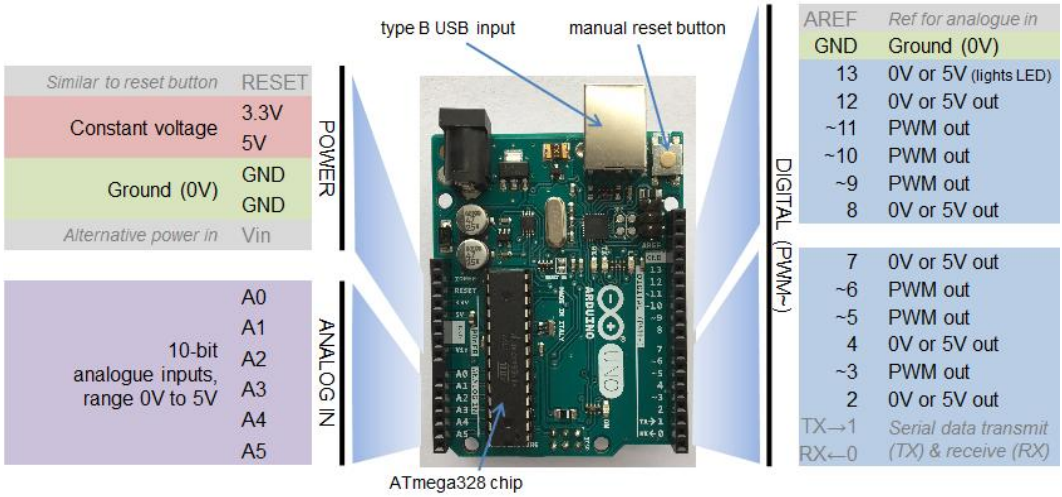


Figure 3 - Plan view of Arduino UNO board annotated with I/O pin assignments.

The UNO has two banks of input/output pins, one on each side of the board, as shown in Figure 3, as well as a manual reset button which (usefully!) can be used to restart the UNO's on-board program.

Pay no attention to those greyed out in the figure (RESET, AREF, Vin, TX→1 and RX←0), as they will play no part in this experiment. The 5 V and 3.3 V pins and the analogue inputs may well also not be needed, but you will be using a ground pin(s) as a reference level, and you will certainly require the voltage-out pins (illustrated in blue, on the right-hand side). These pins can essentially be separated into two types. The first is a high-low or binary type, which may be made to output only 0 V or 5 V (pins 2, 4, 7, 8, 12 and 13). Of these, pin 13 is special, as it is associated with an adjacent orange light-emitting diode (LED) which indicates whether the pin is high or low. The second type of pin, numbers 3, 5, 6, 9, 10 and 11, has a pulse-width modulated (PWM) output; at any given time, they must output either 0 V or 5 V, but by varying the duration of the two states when in a pulsed mode, they can be made to average an arbitrary value in between (allowing for 8-bit, *i.e.* 256-state, precision). Figure 4 shows two traces in PWM mode, the upper having a mean value around 2 V, the lower 4 V. The frequency of variation is around 975 Hz and cannot be adjusted. The PWM pins may also be operated as ordinary HIGH/LOW.

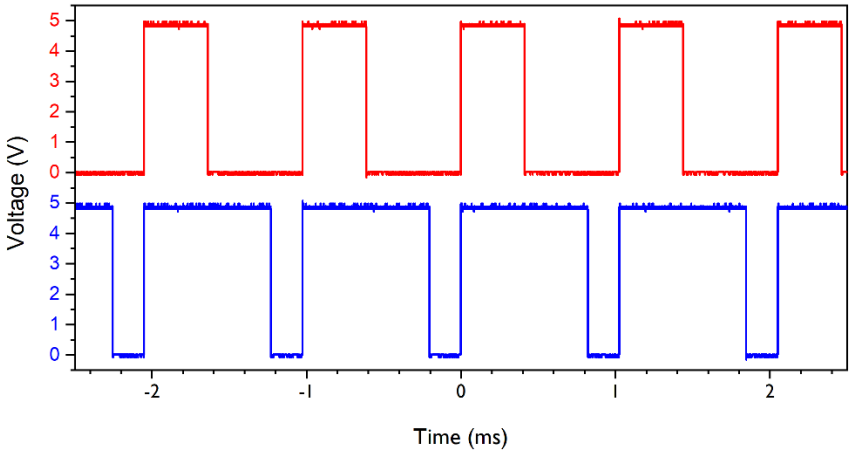


Figure 4 - Pulse-wave modulation (PWM) output voltage variation of an Arduino UNO for set mean voltages of approximately 2 V (upper) and 4 V (lower).

2.3.2 Programming the UNO



Figure 5 - The Arduino IDE.

Note: In this manual, Arduino (and Python) code will be shown `like this`*. File names are displayed* `like_this`*. It may be useful sometimes to copy and paste from the electronic version of this document.*

A detailed description and programming reference is available offline from the IDE's 'Help' menu, or online (Arduino AG, n.d.)*, though the notes in this manual are intended to be sufficient for your purposes. You might find the example sketches built into the Arduino software informative if you're stuck.*

You are provided with a USB type A to type B connector cable which allows you both to power up the UNO and to program it. Connect the cable from laptop to UNO and open the Arduino Integrated Development Environment (IDE) software (Figure 5). In this experiment, you will not be expected to write extensive code. The format of the software is very simple and is based on the C programming language. Each program, known as a 'sketch', contains two functions (declared by the call `void`, so-named as no value is returned by the function); `setup()` executes first, and just once, and can be used for things like defining the data transfer rate or setting up the pin output, and `loop()` executes infinitely, doing whatever it is the program is designed to do. You may also declare variables or constants before, and outside of, these loops.

Do I need to learn to program the Arduino?

Simply, no. The Arduino sketch (program) provided, `phase_shift` (found in the folder "E1 E2 PLL files master"), which creates two digital pulse chains with a fixed phase offset between them, may be all you will need. You may well want to adapt it a little to suit your purposes, but achieving that should be fairly easy.

If you are keen to know more about programming the Arduino, read the more detailed notes in Appendix A1.3, or search for tips online. It will help your understanding, and you might think up some nice ideas to try, either in the phase offset experiment or later on.

If not, read on.

2.4 Python

You will inevitably have at least a working knowledge of the Python programming language, for which https://www.python.org/ provides a comprehensive reference. Python will interrogate functions provided by the PicoScope Software Development Kit (SDK), a free library provided by Pico Technology which bridges the gap between Python and the language which drives the PicoScope hardware, C++. See Section 2.4.1 for details.

It is possible that important components will be absent (you may encounter a missing matplotlib or picosdk), in which case installing them ought not to be too difficult (perhaps take a look at Section A1.2), but ask if you encounter problems. Another issue is the failure to run the code successfully from your code editing/executing package (*e.g.* Visual Studio Code, Jupyter Notebook or your OS Terminal); again, ask.

2.4.1 Picoscope Software Development Kit

The PicoScope SDK is available for download at https://www.picotech.com/downloads via the menu option for PicoScope 2406B. It should already have been installed on the lab laptops, but in case it has not, or if you prefer to use your own machine, it is – hopefully – not too tricky to install. On a Windows 11 machine, open the "Settings" app, then "System → About" to check the system type (likely "x64" rather than "x86"), then download and install the appropriate (stable) PicoSDK, being very careful that you've followed the options for the 2406B rather than for another PicoScope model. You will see that there is an SDK available for Mac, but beware – I have NOT tested Mac OS, so attempting to use your Mac machine for this experiment is not recommended.

2.4.2 Python Scripts Using the SDK

It is hoped that coding should be quite simple for this experiment, but also that those who wish to complicate matters should be free to do so. You are provided with a few Python scripts which – hopefully – will enable you to access your PicoScope without modification if your computer is correctly configured.

The most complicated is `pll_ps2000a.py`, which is essentially a library of the functions you will need. I recommend NOT modifying this file, but those of a more adventurous nature may do so at their own risk (you should have been provided with an electronic version of the PicoScope 2000 Series (A API) Programmer's Guide for such foolishness – if not, it's at the PicoTech website link in Section 2.4.1).

The simplest script is called `control.py` and is an *example* of the type of program you will use to run your experiments. You would very much be expected to do some programming to adapt this file to your needs, or indeed to create your own files, but not too much programming expertise is needed to get it working in a sophisticated way.

Also included is `control_phase.py`, which is a simple program which links to the Arduino sketch `phase_shift`; we will use that combination in studying what happens when two signals are out of phase by a fixed amount – more about that later (Section 6.2). Our last file, `arduino.py`, is called by `control_phase.py`, and is probably not useful for anything else. It contains some routines for opening, closing the Arduino, as well as some serial read/write functions.

If `pll_ps2000a.py` is to be kept in the same directory as a control program (`control.py`, `control_phase.py` or whatever your program is called), the first line of code in that program should be

```
import pll_ps2000a as picoscope
```

You may prefer to put `pll_ps2000a.py` into a sub-folder; let's call it "library". In that case, the first code in your control program should be

```
import library.pll_ps2000a as picoscope
```

There are standard practices for publishing and distributing Python libraries for installation, but there are not used here in the interests of simplicity and ease of editing; apologies to advanced programmers.

Having got that sorted, executing `control.py` should fire up the PicoScope and start a graphical window. If it does not, and if basic checks don't help, ask your Demonstrator or Head of Class.

Once you're over that hurdle, common sense (and, of course, your undoubted intelligence) should be all you need to automate your PicoScope to investigate your circuits.

Functions in pll_ps2000a.py

You will use a small number of functions in the library `pll_ps2000a.py` (`control.py` is an example of such use). For reference, those functions and the parameters you pass to them are described here (much the same information is available in annotations in the code itself).

`control.py` has *essentially* the following structure (in reality you'll need more stuff):

```
import pll_ps2000a as picoscope
picoscope.picoscope_open()
picoscope.scope_prepare(...)
picoscope.awg(...)
picoscope.scope_monitor()
(time,volts)=picoscope.scope_capture()
picoscope.quick_save(time,volts,...)
picoscope.quick_plot(time,volts,...)
picoscope.picoscope_close()
```

where the ellipses ('…') stand in for a collection of parameters (such as channel ID). Each of those functions (excluding the `import…` command) is found in `pll_ps2000a.py`. Essentially, we import our functions, open the PicoScope, tell it what it is supposed to do/measure, tell the arbitrary waveform generator (AWG) to be a function generator, open a monitor window to check the traces look as expected then confirm that we wish to start the capture, capture the data, save it, plot it, and finally close the unit. Your experiment will differ, but this is a reasonable illustration.

In more depth – and do pay attention to the parameter tables if nothing else:

picoscope_open()

The details of this one are not important, but it needs to be called at the start of your program for obvious reasons. Symptoms of it working are the appearance of the PicoScope splash screen and the familiar hardware clicks. If nothing happens, check the PicoScope is working using the proprietary software, and if still not working, unplug and plug in the PicoScope, perhaps into a different USB port.

scope_prepare()

We need to set up the PicoScope – what channels are we recording on, and what voltage ranges? What trigger will we use? Coupling modes? Does the PicoScope record data before the trigger?

A bare-bones un-triggered command on channel A (only) might look something like

```
picoscope.scope_prepare(capture_time=0.1,samples=1000,a_range=10)
```

with the remaining parameters taking their default values.

parameter	permitted values	default	comments
capture_time	>0 (seconds)	0.1 s	The *desired* duration of the captured trace
samples	up to 33554342	1	Number of individual data points – practical considerations will dictate choice
a_range **b_range** **c_range** **d_range**	>0 (volts), None	None (disabled)	Voltage limit (positive and negative) - may adjust to the next highest hardware setting
a_probe **b_probe** **c_probe** **d_probe**	'xN' where N is a positive number (e.g. 'x1', 'x10', 'x4.2'), or N (e.g. 1, 10, 4.2)	1	Commonly used for 'x10' attenuating probes
a_coupling **b_coupling** **c_coupling** **d_coupling**	'dc', 'd.c.', 'DC', 'D.C.', 'ac', 'a.c.', 'AC', 'A.C.'	'dc'	a.c. coupling adds a high pass filter to remove d.c. signal (but beware - it will distort low-frequency a.c. signals)
a_name **b_name** **c_name** **d_name**	Any plain text (e.g. 'V_in')	'A', 'B', 'C' and 'D' respectively	More useful labelling of column titles in exported data and of legends on python-generated plots.
trigger_channel	'A', 'B', 'C', 'D', 'a', 'b', 'c', 'd', '0', '1', '2', '3', 0, 1, 2, 3, None	None (disabled)	Channels A to D correspond to numbers 0 to 3
trigger_level	any in range (volts)	0	Voltage threshold for trigger
trigger_direction	'rising', 'falling', 'RISING', 'FALLING', 'Rising', 'Falling', 'R', 'F', 'r', 'f', 'up', 'down', 'UP', 'DOWN', 'Up', 'Down', 'U', 'D', 'u', 'd'	'rising'	PicoScope triggers on a rising or falling edge
trigger_delay_samples		0	After trigger, PicoScope waits this number of sample intervals before recording
trigger_pre_percent	0-100 (percent)	0	How much of the recorded trace should precede the trigger (n.b. time=0 at trigger)
trigger_timeout	≥0	0	If no trigger in this time (seconds), PicoScope triggers anyway – zero to wait indefinitely
verbose	True, False	True	'False' to suppress output

awg()

The arbitrary waveform generator (AWG) does more or less what the name says, creating in principle any shape of waveform you like, with output at the fifth (right-hand) output terminal of the PicoScope. However, in this implementation we limit it to acting as a simple function generator with a selected offering of standard forms. We are investigating a digital circuit, so the most likely choice is square wave. The AWG produces very much quicker jitter-free pulse chains than the Arduino can (have a go!), so getting used to it is highly recommended. One drawback is that it can only produce voltages between ±2 V, and you may well need to configure an op-amp circuit to produce signals at 0V/5V digital logic levels. You should be able to

find a suitable op-amp in the lab (see Section 6.1 - ask if not, or indeed if you have problems building your amplifier).

One very simple example of using this function (for a steady 1 kHz 0 V→5 V digital signal) is

```
picoscope.awg(peak_to_peak=5,offset=2.5,frequency=1000)
```

and once again any parameters not specified are assigned their default value.

parameter	permitted values	default	comments
`waveform`	'sine', 'square', 'triangle', 'ramp_up', 'ramp_down', 'dc' (case/underscore insensitive)	square	
`peak_to_peak`	≤4 (volts)	2 (volts)	AWG produces voltages in the range ±2V
`offset`	≤2 (volts)	0	
`frequency`	$\leq 10^6$ (s^{-1})	1000 (s^{-1})	
`finish_frequency`	$\leq 10^6$ (s^{-1})	None (no sweep)	
`increment`	$\leq 10^6$ (s^{-1})	1 (s^{-1})	
`dwell_time`	>0	1 (s)	
`sweep_type`	'up', 'down', 'updown', 'downup' (case insensitive)	'up'	
`operation`	'normal', 'whitenoise', 'random' (case insensitive)	'normal'	'whitenoise' ignores sweep settings, 'random' jumbles the order of sweep frequencies

scope_monitor()

Opens a window showing continually-updating (when triggered) trace data. No parameters are passed to the function, and, when closing the window, it will return `False` if the user has clicked the force-close "x" button at top right, or `True` if the user clicks anywhere else on the window. Typically, if `True` is returned, the program may then close the monitor window and proceed to the "real" data capture; if `False`, the program may exit to avoid saving unwanted data files. The user is of course welcome to choose to omit this function entirely (for example to avoid endless mouse-clicks when taking a long series of traces) or to ignore the value it returns.

scope_capture()

This one executes the measurement, waiting first for the trigger conditions (if set) to be satisfied. It returns `(time,volts)`, where `volts` includes all four voltage arrays (even if one or more channels is disabled); `(time,volts)` is thus effectively `(time,(volts_a,volts_b,volts_c,volts_d))`.

The function will have converted buffer values into true voltages based on what you told the PicoScope to do in `scope_prepare()`.

quick_save()

You may prefer to write your own data-saving routine, but this basic one is provided to save you the bother. It will convert instances of the `#` character in the file path to a date stamp of the form `YYYY-MM-DD_hh-mm-ss_uuuuuu` (in order - year, month, date, hours, minutes, seconds, microseconds). If needed, the function will also create any folders and sub-folders. You might be clever and do some coding to have the folder name vary programmatically, but for obvious reasons the `#` conversion method is not a good choice for that! Be careful in defining your path, in case files are written to

unexpected places. The path should include the file name, as is shown in this example (note that `time=time,volts=volts` must always be included in the parameters):

```
picoscope.quick_save(time=time,volts=volts,file_path=f'./data/monday_#.csv')
```

(The f above signifies an *f-string* in Python, allowing you to include variables in your string - I have not done so here, but be familiar with f-strings!)

parameter	permitted values	default	comments
`time`	`time`	`time`	always include
`volts`	`volts`	`volts`	always include
`file_path`	Any valid	'./default_data/#.csv'	'#' is replaced by a time stamp
`delimiter`	Any sensible symbol	`,` (comma)	tab or space, for example
`verbose`	'True', 'False'	True	'False' to suppress output

quick_plot()

As you might expect, this function draws the data you have captured on a graph. It will require you to close it ("x" button at top right only, to avoid unwanted closure) before your program continues. Specifying the parameter `file_path` (as ever, including the file name) will save the image to a graphics file, where the extension ('.png', '.jpg', '.pdf' and others) will determine the required file format automatically (again, `#` is converted to a timestamp).

If you call the function and include the parameter `display=False`, the plot will not be displayed at all and the program will continue – this may seem silly, but if you've chosen to save the graphics, doing this will allow you to generate graphical plots for multiple data sets without endlessly closing graph windows.

An example might be

```
picoscope.quick_plot(time=time,volts=volts,file_path='./img/#.png')
```

parameter	permitted values	default	comments
`time`	`time`	`time`	always include
`volts`	`volts`	`volts`	always include
`file_path`	Any valid	'./default_data/#.csv'	'#' is replaced by a time stamp
`delimiter`	Any sensible symbol	`','` (comma)	tab or space, for example
`verbose`	'True', 'False'	True	'False' to suppress output

picoscope_close()

It is good practice to use this function (which takes no parameters) at the end of your program. Be careful not to close your PicoScope when you don't want to, for example when taking multiple traces. You could open → measure 1 → close → open → measure 2 → close if you like, but open → measure 1 → measure 2 → close is way more efficient.

Background to the Experiment

3 Introduction to Phase-Locked Loops

A phase-locked loop circuit employs feedback to maintain a local oscillator in synchronism with an incoming signal. Its wide applications include signal recovery and demodulation, and frequency multiplication. This experiment investigates the operation of the circuit and the feedback conditions necessary to achieve both stable operation and good response to changing input signals and considers examples of the applications for the circuit.

3.1 The Experiment

The experiment involves the assembly of electronic circuits which allow investigation of an integrated circuit, the HEF4046B chip ('4046' for short). Circuits will be constructed on a prototyping board, consisting of a 'breadboard' - a push-fit matrix with interconnecting tracks - and a power supply, upon which circuits may be assembled and modified non-permanently. In some cases, the circuit will be complicated and will require care and attention to detail. It is generally very worthwhile considering carefully the positioning of the circuit before starting, and taking steps to make the circuit easy to debug by both experimenter and Demonstrator, for example by using a colour code for the wiring and by using appropriate lengths to avoid a 'rat's nest' of tangled wires. Traces will be observed on a PicoScope using Python or, if appropriate, the proprietary PicoScope software. If you have any hardware or software problems it is quite possible that the experiment itself is flawed, so please raise the issue once you have assured yourself it is actually a thing. Basic instructions for the breadboard are given in Appendix A1. Whenever in doubt, have a think, then *ask*.

A considerable part of your lab time will be taken up in assembling these circuits correctly. However, the class is not an exercise in the completion of the greatest number of manual tasks in the given time; the time taken once the circuit is assembled, when you will be studying the behaviour of your circuit and its optimisation (and corresponding write-up time), is by far the most important time you will spend. It is by no means a problem if you do not complete all of the applications in this manual; in fact, it may be a problem if you *do*! A report with good experimental techniques and careful analysis based on a subset of the available tasks will attract far better marks than one which merely ticks the boxes - getting all of the circuits to work with no further thought. As an indication, very good marks can be obtained by completing Section 6. if your experimental approach and analysis is of a high standard, but the best marks will be gained by, in addition, covering some of the applications in Section 7.

This manual gives the principles of phase-locked loops in Section 4 and specifics of the applications in Section 5. Your first experiments will come in Section 6, but you would be well advised to be familiar with the principles before plugging in your first component.

Finally, you are learning the art of experimental work as well as its science. If you have a bright idea for a brief experiment which is both relevant and potentially informative, you are encouraged to include it.

4 Phase-Locked Loop System Operation

4.1 Feedback and Stability

The phase-locked loop circuit is a feedback system, in which comparison is made between the incoming signal and the output signal from the voltage-controlled oscillator to produce an 'error' signal which represents the phase difference between them (Figure 6). This 'error' signal is applied to a voltage-controlled oscillator to determine its output frequency. It is often necessary to incorporate a filter circuit, and the characteristics of that filter must be chosen to remove unwanted signal-frequency (and other high-frequency) components from the error signal and to ensure stability and satisfactory response from the system. The filter's optimum response time will be application-dependent and is a key parameter in successful operation. The 'error' signal may in some cases pass through an amplifier (not shown) before insertion into a voltage-controlled oscillator (VCO).

In some applications, the filtered 'error' signal is itself the required output from the phase-locked loop. For example, in frequency-modulation detection the 'error' signal is a measure of the frequency of the incoming frequency-modulated signal and so gives the modulation signal that is to be recovered by the detector.

Analysis to determine the stability of the circuit is given in Appendix A2. It shows that the basic system, without any filter, is inherently stable.

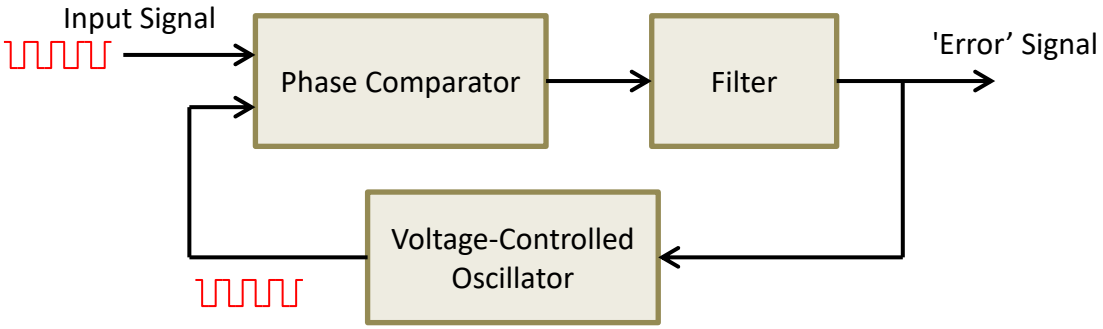


Figure 6 - Basic elements of a phase-locked loop feedback system.

4.2 Effects of Filter Circuits on Stability

With no filter present, the phase-locked loop circuit is very limited in its applications. Real comparator signals vary over a cycle of the signal frequency, and this variation may need to be smoothed out. Additionally, it may be required that the voltage-controlled oscillator provides a steady output even if the input signal is subject to noise or has missing cycles (in clock-signal regeneration, for instance) or if the frequency of the signal applied to the comparator is different from the frequency of the oscillator (as in a frequency multiplier).

In such cases, the obvious approach is to interpose an R-C integrator (Figure 7) to smooth the 'error' signal and to apply the smoothed signal to determine the frequency of the voltage-controlled oscillator.

However, this filtering can affect the stability, as discussed in Section A2; to overcome this problem a variant on the basic integrator can be used, the lead-lag circuit of Figure 8. I should caution you that in most cases this is NOT necessary, and it may well cause you headaches with interpretation (how do I define a time constant?)! In such a case, stick with the ordinary RC filter.

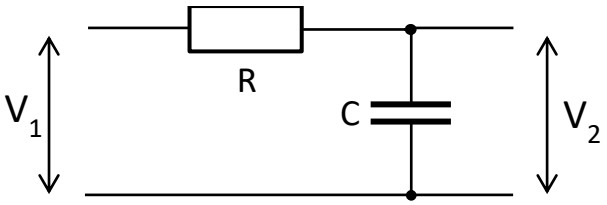


Figure 7 - Integrator using a simple R-C filter.

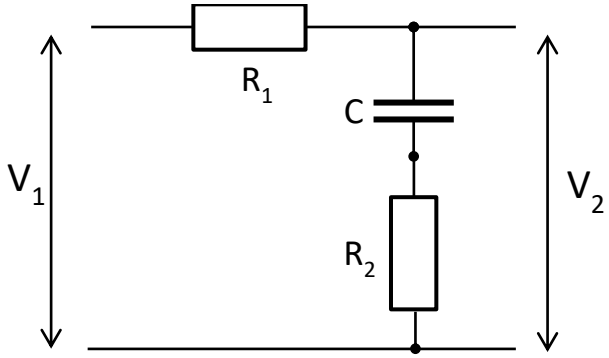


Figure 8 - Integrator using a lead-lag circuit for zero phase shift at high and low frequencies.

4.3 Tracking and Capture

Once the oscillator has locked to the frequency of the input signal, the oscillator frequency will track the input provided that the rate of change of input frequency is not excessive. For systems in which the control voltage for the oscillator does not vary significantly during any cycle of the oscillator output, tracking will occur provided the input signal remains within the range that the voltage-controlled oscillator can provide for the available control voltage swing; if the control voltage varies over a cycle, more restrictive conditions may apply.

The capture process by which the local oscillator initially locks onto the incoming signal is relatively complicated, and capture is not necessarily possible for the full range of frequencies within which tracking is possible. Both the oscillator frequency in the absence of signal input and the capture characteristics depend on the type of comparator.

5 The CMOS Integrated Circuit Type 4046 Phase-Locked Loop

The venerable HEF4046 phase-locked loop chip has long been at the heart of this experiment. However, there is a long tradition of CMOS manufacturers releasing new generations which are functionally identical to their predecessors. In the case of the HEF4046, we can therefore substitute with the 74HC4046, which has all the same pin-out functions as the 4046, with just one small difference - a third type of phase comparator output, replacing the HEF4046's Zener output on pin 15. We will not be investigating this third comparator this year.

There *should* be enough HEF4046s to go round this time, and I suggest you use them in case of unforeseen issues. On the other hand, I would be interested to see whether the new version performs relative to the old, so the brave may opt straight away for the 74HC4046. In the event of all of you needing to upgrade, everything will be fine. Or if it isn't, either raise the issue straight away or, again if you're brave, characterise the difference.

For those interested, there is a micrograph of the HEF4046 chip in Appendix A5, along with some tips for recognising the individual circuit elements. We will concentrate on recognising its behaviours in a more block-diagram, or black-box, way.

5.1 General Outline

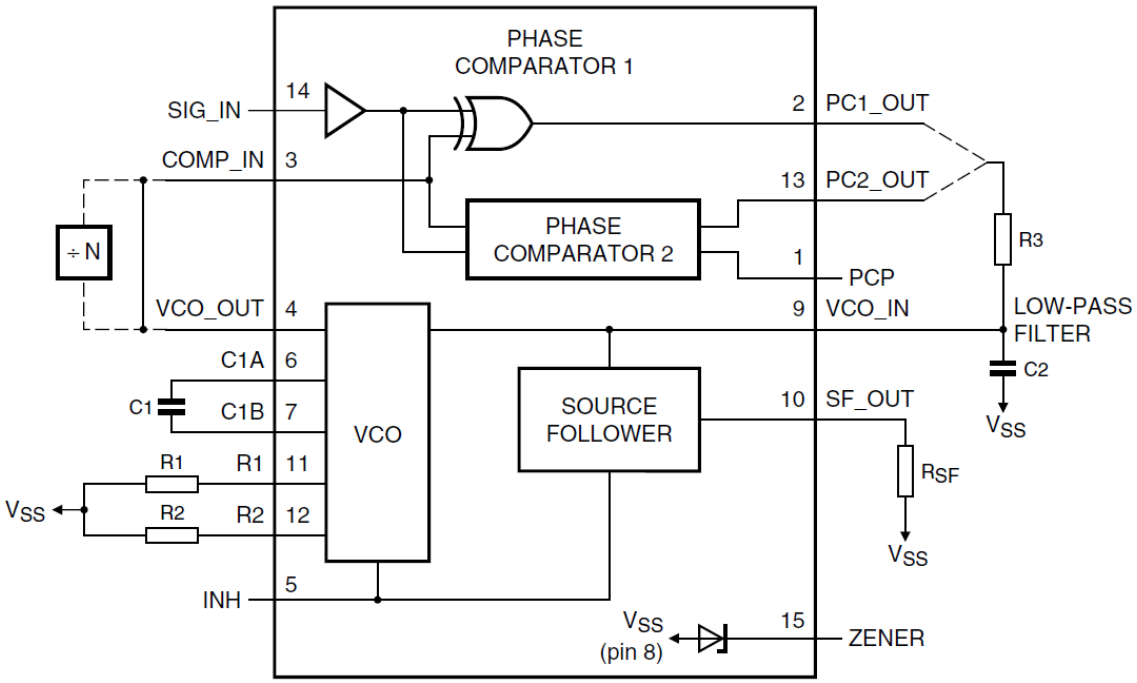
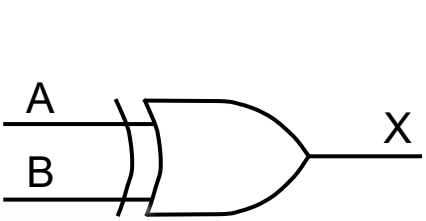


Figure 9 - Schematic of the HEF4046B phase-locked loop integrated circuit (NXP, n.d.).

The 4046 CMOS (complementary metal-oxide-silicon field-effect transistor) integrated circuit incorporates all necessary active elements for a phase-locked system, requiring the addition of only a few passive components. It contains a voltage-controlled oscillator and two different phase-comparator sections, with common input circuits (Figure 9). The input amplifier for the 'signal' channel can accept small signals as well as CMOS logic-level signals, but for this experiment only logic-level signals will be required. With a supply voltage of 5 V, logic level '1' is about 5 V and '0' about 0 V. Pin connections are shown geographically (with the notch or dot on the package giving orientation) in Appendix A3.

5.2 Phase Comparator Type 1: EXCLUSIVE-OR Circuit



		A	
		0	1
B	0	0	1
	1	1	0

Figure 10 - EXCLUSIVE-OR circuit symbol and truth table.

The EXCLUSIVE-OR circuit provides an output which is high when only one of the inputs is high, so that the truth table for the output X is as shown in Figure 10.

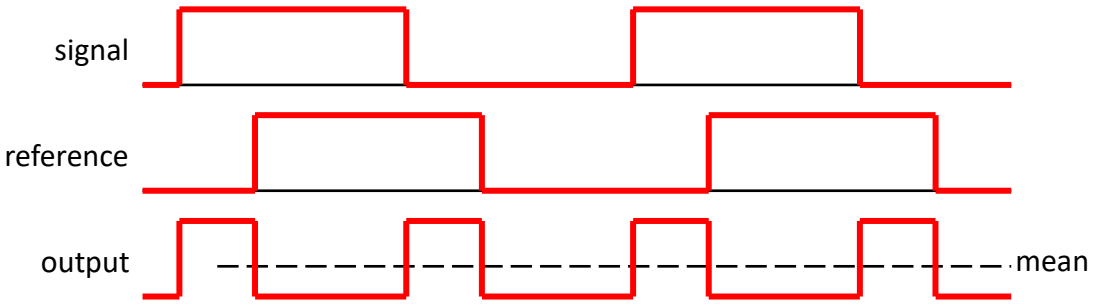


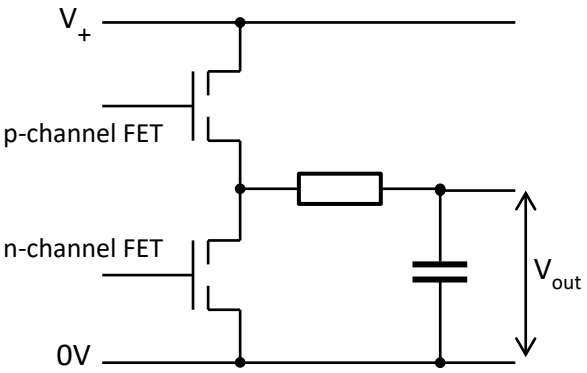
Figure 11 - EXCLUSIVE-OR circuit: idealised signals.

With two identical input signals the output is always low, whilst two signals with the same frequency but different phase will give a rectangular-wave output signal (Figure 11). This signal has twice the frequency of the input signals, with its mark-space ratio dependent on the phase difference; thus, the mean level of output is proportional to the phase difference between the inputs. Note that when the mean output level is at its mid value (because its mark-space ratio is unity) the input and the reference signals are 90° out of phase.

5.3 Phase Comparator Type 2: Edge-Triggered Circuit

The second phase comparator produces output pulses which depend on the rising and falling edges of the input reference signals. These pulses are intended to be applied to an R-C integrator circuit, external to the 4046 chip, so that the voltage on the capacitor can be used as the input to the voltage-controlled oscillator. When the input and the reference signals are at the same frequency (with phase difference constant), the capacitor voltage must be held constant; for the complete phase-locked loop, this voltage would be the voltage for which the oscillator frequency is equal to the input frequency. If the input and the oscillator signals are at different frequencies, the capacitor voltage must be driven in the direction that would bring the oscillator into synchronism.

The requirement for increasing the integrator voltage, or for decreasing it, or for maintaining it constant is achieved by driving the R-C integrator from a three-state output stage by which the capacitor voltage can be driven higher or lower, or the capacitor can be isolated (Figure 12). The state of the output stage is determined by a logic circuit whose operation is indicated by the state diagram in Figure 13; each circle in this diagram represents a state that the logic circuit can occupy, whilst the positions of the circles and their shading indicate the corresponding states of the output stage. Take some time to understand Figure 13 fully.



p-FET	n-FET	Effect on V_{out}
OFF	OFF	V_{out} cannot change
ON	OFF	V_{out} increases towards V_+
OFF	ON	V_{out} decreases towards 0V
ON	*ON*	*(This state is never applied by the control circuit)*

Figure 12 - Schematic of three-state output stage driving an R-C integrator; in the 4046, the two FETs are incorporated in the integrated circuit, whilst the capacitor and the resistor are external.

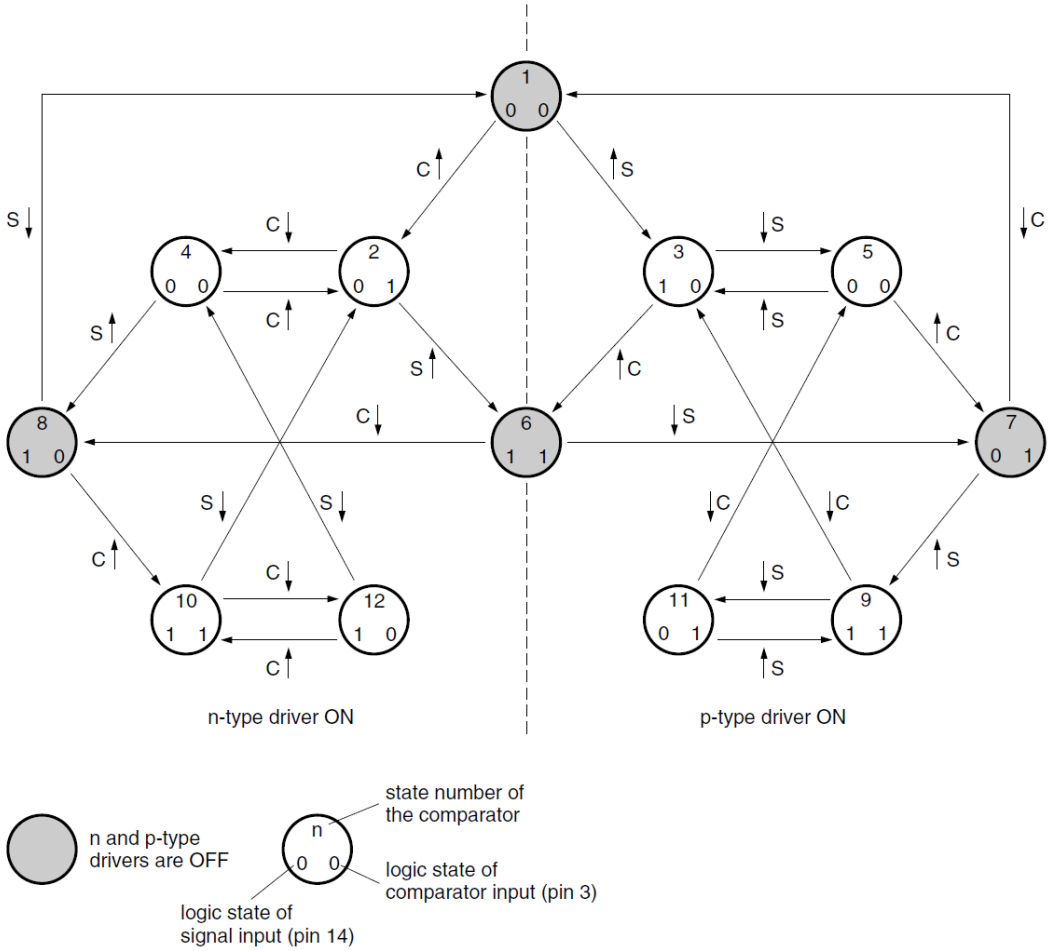


Figure 13 - State diagram for type-2 comparator in Philips HEF4046B. n- and p-type 'drivers' refer to FETs of the type shown in Figure 7; both are 'off' for the states shaded grey, whilst the unshaded states in the left and right halves represent, respectively, n-FET 'on' and p-FET 'on'. (NXP, n.d.)

Transitions between the logic-circuit states occur according to the rising and falling edges of the input and reference signals. For example, if the input is at the same frequency as the reference signal, and leads it

in phase, then their edge transitions will follow the cycle .... C↓, S↑, C↑, S↓, C↓, ... and the corresponding state-number sequence would be .... 7, 1, 3, 6, 7, ... so that the output stage would have both transistors off except for the period between S↑ and C↑, during which the integrator would be driven positive, thus tending to increase the local oscillator frequency and so tending to pull it into phase with the incoming signal. In general, the edge transitions follow a sequence which results in output-stage transistors being turned on to give an appropriate change to the integrator voltage; for instance, if the signal frequency is initially four times the reference frequency, a cycle such as .... C↓, S↑, S↓, S↑, S↓, C↑, S↑, S↓, S↑, S↓, C↓, ... might result, with a state-number sequence .... 11, 5, 3, 5, 3, 5, 7, 9, 11, 9, 11, ... which would hold the p-channel FET on for most of the time, tending to drive the integrator voltage higher. From arbitrary starting conditions the state sequence will be less regular and will change as the oscillator frequency is pulled into synchronism.

The HEF4046 and 74HC4046 chips both operate in the same way, though the data sheets may seem different.

5.4 Voltage-Controlled Oscillator

The voltage-controlled oscillator in the 4046 uses drive circuits similar to CMOS output stages to drive an integrator, so that switching occurs when the voltage of an external capacitor C reaches a particular value. It is probably similar to Figure 14 (the seemingly strange multiple NOT gate chains at A_1 and A_2 do have a subtle use in delaying the signal, but the reasons for this are unimportant here).

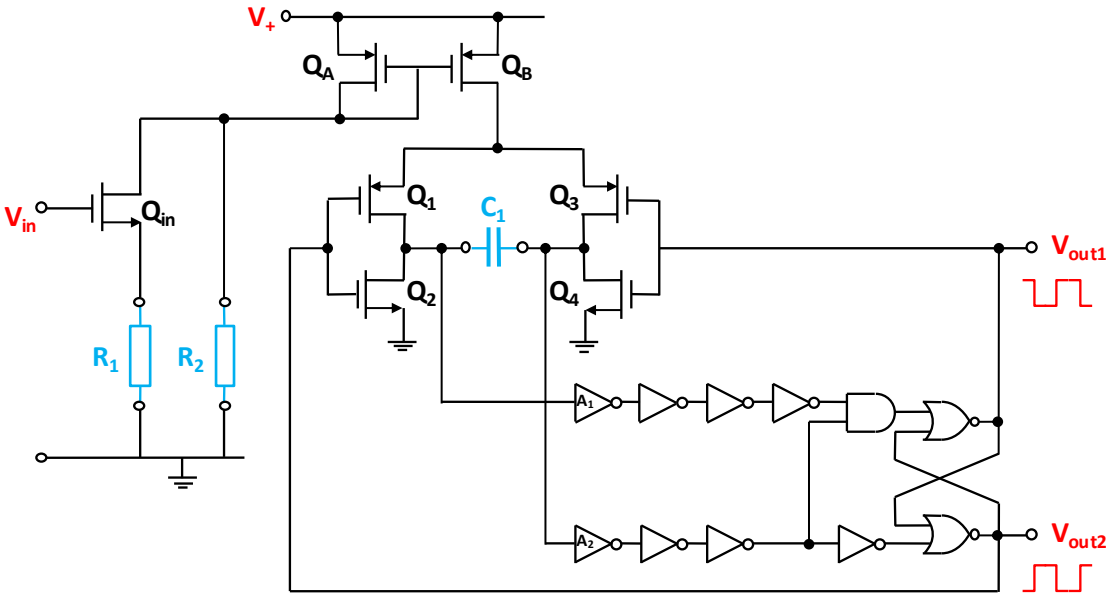


Figure 14 - Plausible arrangement of the voltage-controlled oscillator. C, R_1 and R_2, highlighted, are discrete components external to the integrated circuit. Devices Q_i are MOSFETs of two types (see Section 5.4.1).

The capacitor is charged alternately with opposite polarities. When the output voltage reaches a reference value, one of the comparator circuits A_1 and A_2 trips, and the charging current drive is reversed. The input stage is a current-mirror circuit so that (if the two FETs Q_A and Q_B are identical) the charging current I_c will equal the input current (see Section 5.4.1). As shown, the total input current is the sum of the two terms, a fixed term set by the external resistance R_2 and a variable term set up by the input voltage V_{in} and the resistance R_1.

5.4.1 MOSFETs

Figure 14 contains seven individual MOSFETs (metal-oxide-semiconductor field-effect transistors – three-terminal devices in which the current flowing between two terminals is controlled by the voltage applied to the third. Understanding these to any particular degree of sophistication is not important, but some basic aspects will help with understanding. Figure 15 shows in very simple terms some key features.

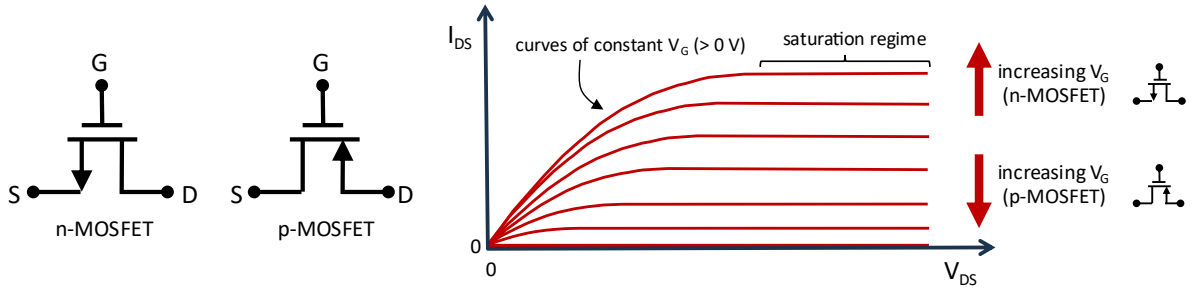


Figure 15 - Simplified description of MOSFET principles and circuit symbols. All voltages are taken to be positive (or zero) in our digital electronics (0 V to 5 V) application. Note the arrows on the circuit symbols denoting the MOSFET type; "n" and "p" indicate the doping type in the channel between drain ("D") and source ("S"). The variation of gate ("G") voltage V_G has an opposite effect on the two varieties at a given drain-source voltage V_{DS}; more positive voltage causes the drain-source current I_{DS} to increase in an n-MOSFET but to decrease in a p-MOSFET. In effect an n-MOSFET may be considered switched "on" with a digital "high" V_G and "off" with a digital "low" V_G., with the converse true for a p-MOSFET.

The first thing to note is that there are two distinct device types (identified by the direction of the arrow on their circuit symbol), and you will see both appearing in Figure 14. The MOSFETS labelled $Q_{1,2,3,4}$ should be thought of as simple switches, with +5 V and 0 V gate voltages turning them either on or off. Is it clear how the current flows through that part of the circuit (and to/from the capacitor) as the logic circuit at lower right alternates?

MOSFETS Q_A and Q_B have a subtler operation, and they provide the current-mirror feature which causes the currents I_c down both sides of the VCO circuit to be equal (if Q_A and Q_B are identical…!). The clue is that both will largely operate in the *saturation region* of Figure 15, though not necessarily with identical V_{DS}. What does I_{DS} depend on here? *(Hint: not much!)*

Lastly, we have the input MOSFET to the VCO, labelled Q_{in} in Figure 14. Clearly this can be thought of as a variable resistance combining with the external resistors R_1 and R_2, and it is very much NOT a two-state switch. How does it provide a sensitive way to control I_c?

5.5 Quiescent Conditions and Signal Capture

The two types of comparator behave differently both in the absence of an input signal and in their response when a signal is applied. In some cases, the sudden application of too high an input frequency can result in the local oscillator running at a sub-multiple of the input frequency rather than locking into synchronism; this can occur even for a frequency at which the oscillator would operate in synchronism if the input frequency had initially taken a lower value and had then been raised sufficiently slowly. Thus, the frequency range over which the signal capture can occur is not necessarily the same as the range over which oscillation is possible, and the local oscillator may fail to track the input signal.

For the type-1 comparator in the absence of input signal, the exclusive-or gate output is the same as the reference (local-oscillator) signal. if the local-oscillator mark-space ratio is unity, then the mean comparator signal is mid-way between the limiting values, and thus with the feedback loop closed the local oscillator frequency is driven to the middle of its frequency range (neglecting non-linearities).

In the corresponding case for the type-2 comparator, only reference (local-oscillator) edge transitions are present. This is equivalent to the reference frequency being very much greater than the input signal frequency (and so with the logic circuit states alternately 2 and 4, or 10 and 12), and thus the local oscillator frequency is driven to its lower limit.

The type-2 comparator permits capture over the full frequency range of the oscillator, as can be seen from consideration of the logic sequence in Figure 13. for an input frequency which is several times greater than the oscillator frequency. It provides a drive signal which would tend to drive the oscillator frequency in the required direction, irrespective of whether (or how rapidly) the oscillator responds.

However, the use of the type-1 comparator can enable the oscillator to lock onto a signal in such a way that the input and oscillator frequencies are harmonically related and thus capture over the full oscillator range may not be possible.

There is a further problem with the type-1 comparator. Consider the operation of the circuit with an integrator with a very long time constant. Before the feedback loop is closed, the detector output will have a mean value of one half the peak level, irrespective of the frequencies of the two square wave signals, if it is averaged over a sufficiently long period. Therefore, when the feedback loop is closed the input to the voltage-controlled oscillator will remain constant and capture will not occur. Thus, for capture to occur, the filter time-constant must be sufficiently short for the local-oscillator frequency to change within the period over which the comparator output varies.

5.6 Noise and Interference

Susceptibility to noise or interference is a function of the comparator type. With the type-1 comparator, the effect is immediate and occurs only during the presence of the interfering signal. The type-2 comparator can take some periods to recover from interference, and this can be shown by considering the output waveform from the logic of Figure 13 with different starting conditions.

The Experiment

6 Experimental Characterisation of the Phase-Locked Loop Integrated Circuit

The characteristics of the elements of the 4046 CMOS integrated circuit should be investigated and these elements combined to form a complete feedback system. Response and stability should be investigated for the basic phase-locked loop, which is then adapted to meet specific functional requirements.

Even when a method is suggested in this manual, it is up to you to decide how best to approach the experiments. You have the electronic apparatus discussed earlier, but feel free to build any circuitry, or to make use of the AWG function of the PicoScope VI. You may wish to take steps to smooth the signal using suitable capacitors to ground.

Since you will have differing backgrounds and skills, you may approach the experiment according to your abilities; some may focus on sophisticated programming and analysis, some on their insight into analogue or digital electronics. Programmers should remember that no credit is obtained by applying flashy features which reveal nothing about the system under test, and electronics buffs should bear in mind that this experiment should make at least some use of automated measurement.

However you approach it, you must always remember to apply the usual rules of scientific experimentation; conclusions must be backed up by the available data, and validity demonstrated. This should be a characteristic shared by all.

6.1 Useful Circuits

You will quickly realise that applying a voltage and/or measuring a current is not actually trivial with the hardware provided. Here are some ideas. You may not need them.

D.C. voltage source

Most straightforward for the purpose of supplying a d.c. voltage is a potentiometer (Figure 16), a simple potential divider. The resistance between the two front pins in the picture is constant, but a third pin at the rear sits at an intermediate position (potential) which may be adjusted by tweaking the front dial with a screwdriver. This is simple, but has the significant drawback of not being programmatically alterable.



Figure 16 - Potentiometer.

A more sophisticated supply uses the arbitrary wave generator (AWG) port of the PicoScope (see Section 2.4) to provide a voltage. This however is limited to between -2 V and +2 V, so you are likely to need an amplifier – an inverting op-amp circuit with gain -2.5 will take voltages between 0 V and -2 V and convert them to 0 V to +5 V, for example. This is fully programmable and suitable for data sweeps, but do check that the output is as expected.

As touched on in Section 2.3, the Arduino has a pulse-width modulation (PWM) capability for which the proportion of time spent at "high" (+5 V) rather than "low" (0 V) may be selected. This may be made approximately d.c. using an integrator (probably an RC filter). This is also a good option for computer control, but the integration feature also makes it slow to change, so suitable choice of time constant and experiment timings is very important.

There may be a good reason, usually involving output impedances, to pair any of these solutions with an op-amp circuit; read on.

Amplifiers

The AWG output will usually require an amplifier, and an op-amp circuit is likely to be really good at providing this. You are provided with some TL071-type op-amps, identical to those in the IB Teaching Labs[1]. You should be able to make use of them without fuss, but be aware that they run off a ±15 V supply. The variable power supply on the breadboard (three right-hand outputs) can manage this, but that supply is independent of your usual 0 V/5 V supply, so wire the 0 V outputs together.

A very useful reason for using an op-amp is to deal with inadequate output impedances; for example, if you use an RC integrator to average an Arduino PWM output, you might find that once connected to a circuit, the load will cause your supplied voltage to drop. This calls for a "buffer" op-amp circuit (Figure 17). This circuit has unity gain ($V_{out} = V_{in}$) and "transforms" a low output impedance into a high one. An extremely handy thing.

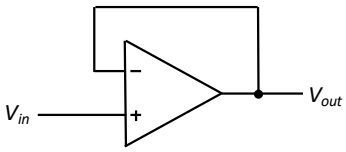


Figure 17 - Unity-gain buffer.

Measuring Current

Unless you're doing something really fancy, the inferring of currents from voltage drops over components is likely to be all you need, though you will have to consider the input impedance of the PicoScope plus leads. If fanciness is your thing, you might like to consider a transimpedance amplifier (Figure 18). This circuit converts current to voltage ($I_{in} = -V_{out}/R$), and simply substitutes for the current's connection to ground. An ideal op-amp draws zero current between V+ and V-, so I_{in} flows via the resistor to the op-amp output. A neat trick, but certainly a niche one in this experiment.

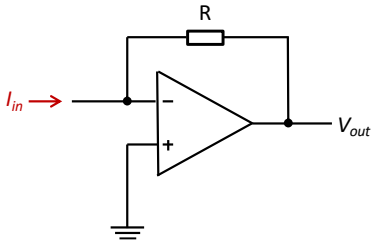


Figure 18 - Transimpedance amplifier.

6.2 Phase Comparator Characterisation - Phase-Shift Supply

To demonstrate the phase-comparator characteristics, it is necessary to have two signals at the same frequency with a controllable phase difference between them. The use of two independent oscillators does not provide adequate control (think why). To facilitate your measurements, you should set up a phase-shifter instrument by which it is possible to shift the relative phase of the signals in either direction, passing smoothly through the condition in which the two signals are in phase. The method should meet another requirement: for compatibility with the CMOS logic circuits: it should provide square-wave outputs, switching between logic levels '0' and '1' (here about 0 V and about +5 V).

Hopefully you will have spotted that your Arduino UNO boards provide a means of applying a suitable signal. You are provided (in folder "E1 E2 PLL files master") with an Arduino sketch named **`phase_shift`** which is able to provide a variable phase difference between two logic-level square-wave signals appearing at output pins of the Arduino. In addition, there is a Python program **`control_phase.py`** which makes it simpler to send commands to the Arduino (you may also use the Arduino's Serial Monitor" window to type commands directly, and this may well be useful during testing (use the PicoScope software to observe what's happening). If doing so, note that the Arduino takes *every other* line of text as a command!). Use this combination, or indeed your own invention, to evaluate the phase comparator's performance and discuss any drawbacks you encounter.

Aside: from bitter experience, though it may be fun to learn to write a sketch which does all of this, there are too many annoyances and unpredictable behaviours in doing so to make it practical in the given time –

[1] Unsurprisingly, as that's where I steal them from.

principally the need to disable interrupts to get reliable timings, which then causes characters sent over the serial link to be lost. `phase_shift` *and* `control_phase.py` *are a reasonable workaround. The Arduino sketch* `phase_shift` *should not usually need editing.*

A general constructional point, to be considered here and throughout the experiment, is the need to ensure that the power-supply lines do not provide unwanted signal paths. Particularly (but not solely) for fast digital circuits, rapid change within an integrated circuit can cause sudden change in the current drawn from the power-supply line and thus (since it is not supplied from a zero-impedance source) in the supply-line voltage. This voltage change can couple into other circuits, causing spurious operation. While this is all unlikely to cause much harm here, a cure is to improve power-line decoupling: in addition to putting a capacitor across the supply at its entry to the circuit board, it is appropriate to put a low-loss capacitor (ceramic or polyester) between each supply rail and ground in the immediate vicinity of each integrated circuit.

6.3 Phase-Comparator Characterisation - Type 1 Comparator

Apply the signals from the phase-shift system to the signal and reference inputs of the phase comparator, and record representative waveforms. What is the transfer characteristic of the comparator, that is, the dependence of the mean output voltage on the phase difference between the inputs?

6.4 Phase-Comparator Characterisation - Type 2 Comparator

To demonstrate the characteristics of this comparator without introducing ambiguity due to the high-impedance state of the three-state output (which is part of the integrated circuit), connect the output to a suitable reference voltage from a high-resistance potential divider, Figure 19. Then the signal monitored at the PicoScope will be held at 'logic 0' or at 'logic 1', provided that one of the transistors is turned on; if both are turned off, then the observed voltage will be that given by the potential divider. Record representative waveforms and estimate the transfer characteristic for the dependence of the mean output voltage on the phase difference. Potential divider resistances should be chosen to give the expected behaviour and **should <u>not</u> be retained in later circuits!** (think - why not?) Something between **10 kΩ and 100 kΩ** should do.

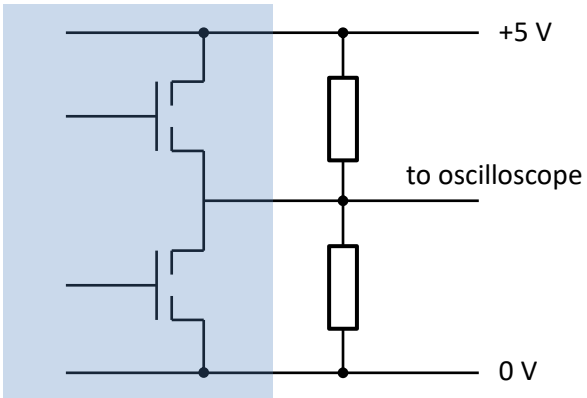


Figure 19 - Load circuit to show the output characteristics of the type-2 comparator. Importantly, the area shaded in blue is part of the phase comparator's on-chip output circuitry, and the transistors are controlled by internal logic (the nature of which is beyond the scope of this experiment); only the resistors are added externally, and they should be removed in subsequent experiments.

6.5 Voltage-Controlled Oscillator Characterisation

The oscillator in the 4046 integrated circuit permits independent control for the centre frequency and for the frequency range that can be covered. This is represented in Figure 14, which is a plausible schematic for the circuit.

What is it about this circuit which makes it a 'current mirror', constrained to supply identical currents to its two branches (if we assume that Q_A and Q_B are identical)? Section 5.4 should help.

The oscillator frequency is directly proportional to the drive current to the capacitor, and that current depends on two resistors and the control voltage. The resistor R_2 sets a standing value of the capacitor drive current and so sets the frequency of the oscillator when the control voltage is zero. The load resistor R_1 for the input source-follower sets the relationship between the control-voltage increments and the drive-current increments. To avoid excessive input currents, neither R_1 nor R_2 should be less than 10 kΩ.

Because the integrated circuit operates with a single power supply of +5 V, it is convenient to choose the free-running frequency to be the frequency when the control voltage is half the supply voltage (+2.5 V) and so to allow for frequency excursions in either direction. This is consistent with the use of the EXCLUSIVE-OR comparator: with input and oscillator signals at the same frequency and with relative phase π/2, the comparator will provide a 5 V peak-peak square wave having a mean value of 2.5 V.

Set up the voltage-controlled oscillator with R_1 = 100 kΩ, R_2 = 100 kΩ and C = 1000 pF, and check that the voltages on each end of the capacitor are consistent with the assumed form of the circuit. **NOTE: The 'inhibit' input must be grounded for the voltage-controlled oscillator to operate!** Does the current-mirror circuit give the same current in each branch (*i.e.*, in the input side and in the capacitor side)?

Explore the dependence of the oscillator frequency on the value of the capacitor and the resistors (for a few values, say 10 kΩ, 100 kΩ and 1 MΩ, and with the resistors omitted to give infinite values), and on the control voltage, so that you will be able to choose values that will give particular centre frequencies and frequency ranges, and that you will know the corresponding values of the coefficient K_O, the constant of proportionality (in the linear regime) between the VCO's input voltage and output frequency.

6.6 Basic Closed-Loop Operation

You may wish to configure this experiment to run under computer control and measurement, but some may prefer to construct plots manually.

Set up the complete phase-locked loop circuit. Choose voltage-controlled oscillator external components for approximately a factor-of-two frequency range which lies within your range of input frequencies; for example, 50 kHz to 100 kHz range if the input signals are to be varied between a few tens of kHz and a few hundreds of kHz.

Close the feedback loop, trying both the type-1 comparator and the type-2 comparator. It may be useful to provide a variable resistor to attenuate the comparator output. Investigate experimentally the stability, lock range and capture characteristics. Consider also the variation with input frequency of the phase relationship between the input and output signals. is it possible to lock onto signals that are harmonically related to the oscillator frequency?

With the current locked to an input signal, look at the waveforms on the capacitor of the voltage-controlled oscillator. Vary the input frequency. Are the waveforms consistent with the assumptions about circuit operation? How do the waveforms relate to the tracking range?

This configuration is particularly important to understand. Investigate it, and make multiple relevant plots and observations to confirm or disprove your thoughts. There are plenty of possibilities, and you may well decide that you do not need to proceed to Section 7!

7 Applications and Investigations

Now we will explore some simple experimental implementations of the 4046 chip. Again, the number of experiments completed is of far less importance than the quality of the investigation. Where there are two (or more) ways to approach the experiment, you need only to attempt ONE.

7.1 Frequency Multiplier

It is often useful to generate a waveform at a multiple of the frequency of a given waveform. For example, to remove the appearance of 50 Hz ripple on a cathode-ray tube display, the line and frame frequencies can be locked to the 50 Hz mains supply. Whilst this is now obsolete as an application for video displays, it remains useful in instruments such as scanning electron microscopes where alternating fields can deflect the electron beam and degrade the image. Many more applications for frequency synthesis exist in computer architecture and consumer electronics.

To implement frequency multiplication with a phase-locked loop, incorporate a divider in the feedback loop, Figure 20, so that the comparator causes locking to the corresponding sub-multiple of the frequency of the local oscillator. Either of the 4020-type counters may be used.

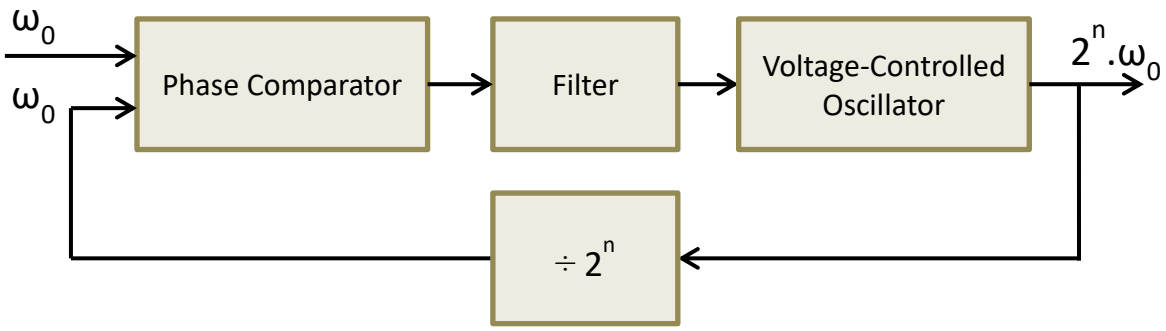


Figure 20 - Use of divider circuit to provide frequency multiplication.

Use the technique to generate a nominal 204.8 kHz signal from a nominal 50 Hz signal (from a signal generator, NOT the mains!). Establish suitable oscillator and filter conditions for this application.

Consider the use of each of the phase comparators available in the 4046 circuit. Consider any unwanted effects that the comparator may introduce, and assess which is better for this application.

7.2 Transient Response and Frequency Modulation

The objective is to use the phase-locked loop to recover a signal that has been frequency- modulated onto a carrier, both to demonstrate the application and to act as a means for observing some transient characteristics for the circuit.

Once again, there are different ways of approaching this experiment, in terms of how to generate a frequency-modulated signal to demodulate. However, bear in mind that the experiment in the last paragraph of this sub-section is only appropriate to a 'breadboard-only' setup, which is described as follows:

It is convenient to use two 4046 integrated circuits, with the voltage-controlled oscillator of one used as the source of the frequency-modulated carrier signal, and the other providing a complete phase-locked loop to act as a demodulator. The arrangement is shown schematically in Figure 21; the output should be taken from the source-follower output of the voltage-controlled oscillator and may require some filtering.

To permit clear display of the signals, it is desirable to arrange both carrier and modulation to repeat in such a way that the composite waveform is itself periodic, for input to the PicoScope.

This requires that the signals are generated from a common source; a divider circuit may be used so that the modulation is a rectangular wave, and both the higher-frequency and lower-frequency parts of the cycle contain the same number of cycles in the carrier frequency.

Optimise the filter for the recovery of modulation waveforms. It is suggested that the carrier frequency should be about 200 kHz, modulated by about ±5% (you may run slower if your setup dictates). Try different frequency-division ratios between the carrier and the modulated waveform to determine the highest frequency that can be used.

The modulation frequency is a square wave; hence the modulator introduces an abrupt change to the frequency of the modulated carrier, which the detector phase-locked loop must attempt to follow. This provides an opportunity to observe the form of the transient response, to an abrupt frequency change, for both type-1 and type-2 comparators. A useful trick here is to add the input and output signals using the 'Tools -> Math Channels' function of the PicoScope software or in Python, and to consider the form of the resultant composite signal; this removes any ambiguity and any possibility of error in the relative phase of the two signals. Taking an external trigger signal from the divider output is useful.

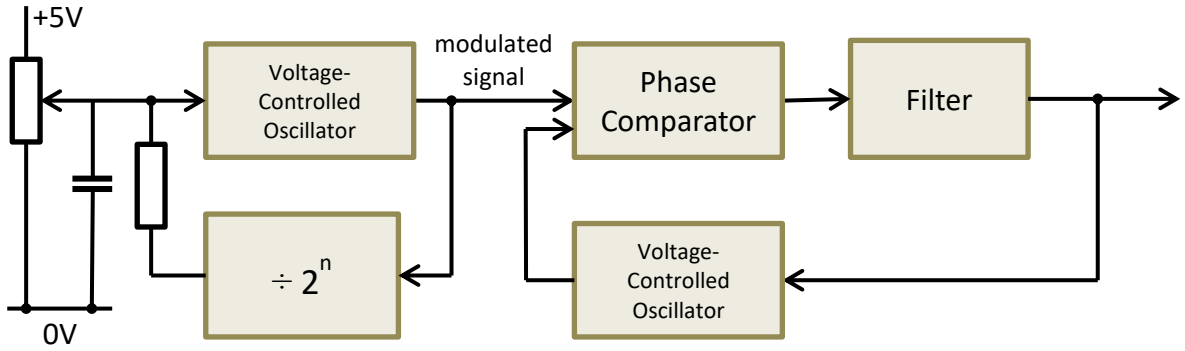


Figure 21 - Test circuit for transient response and for frequency modulation and demodulation.

In a real frequency-modulation system, the modulation signal has a defined bandwidth, narrow by comparison with the carrier frequency; the demodulator is not subjected to an abrupt change of input frequency. This situation can be simulated by adding a capacitor, as in Figure 21, to form an R-C filter at the input of the modulator voltage-controlled oscillator, whilst keeping the remainder of the circuit constant. Investigate the behaviour in this case, with a range of filter time constants.

7.3 Clock Regeneration

Another application for phase-locked loops is for the regeneration of clock signal at the receiving end of a data link in which pulses, synchronised to the transmitting-side clock, are sent down a single data channel. The regenerated receiving-end clock signal must be in phase with the single pulses that are received; however, these received pulses are intermittent because they represent data, and the received pulses may be distorted, and noise may be present (Figure 22).

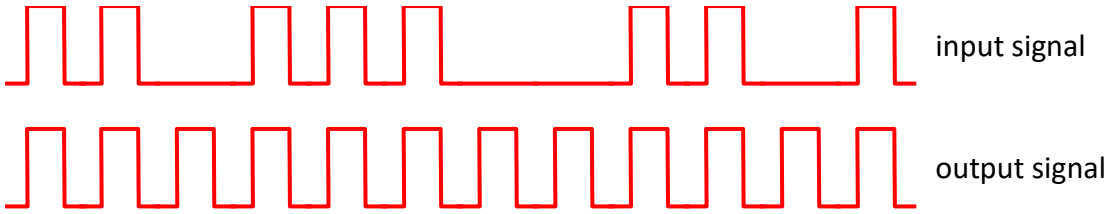


Figure 22 - Pulse train with missing pulses, regenerated to give a complete pulse train.

Characterising Clock Regeneration

You have a choice of two ways to approach this experiment and should attempt only one of them. The first is the breadboard-based method of past iterations of this practical class and is perfectly valid.

For both techniques, you should optimise the filter of the phase-locked loop for the recovery of the 'clock' waveform. The characteristics can also be modified by adjustment of the PLL's oscillator, both in respect of its free-running frequency and its adjustment range. Examine the dependence of the error signal immediately after the pulse burst starts on the difference between the carrier frequency and the free-running frequency, and on the choice of the ratio between the oscillator resistors R_1 and R_2. As for the frequency-modulation trial, consider the results also for information about the transient characteristics of the circuit, and investigate the effects of changing the filter network.

Method 1

A convenient way of simulating a modulated pulse is to use a train of bursts of pulses, with a number of pulses followed by an equal interval in which the pulses are suppressed. The previous circuit can be modified for this purpose, with the output of the divider used to gate the 'carrier' signal, Figure 23. The carrier frequency is set by adjusting the d.c. input to the voltage-controlled oscillator.

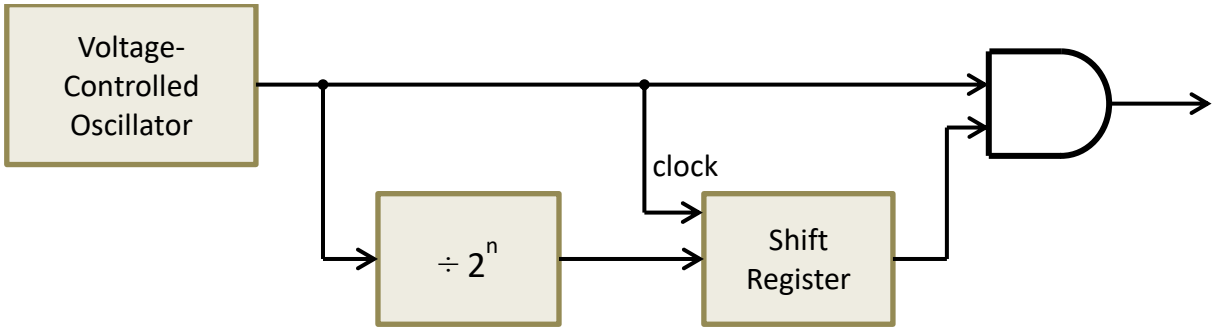


Figure 23 - Circuit for generating a train of bursts of pulses.

To facilitate viewing the waveforms, the modulation waveform can be delayed by a number of cycles using a shift register, enabling the non-delayed waveform to be used for triggering the PicoScope.

Method 2

Whilst providing less of an insight into mixed digital/analogue electronics, a computer-controlled experiment may be extremely powerful, allowing a large data set to be obtained. Use the Arduino UNO to provide a sensible variety of waveforms. As for Method 1, you should prepare a 'gap' in your pulses of a certain duration, though here you may prefer to observe how the detector PLL circuit behaves when a long pulse chain gives way to a long absence of pulses (*i.e.* repetition not necessary). The computer-controlled technique gives you the possibility of varying some parameters as you see fit.

7.4 Input MOSFET Characterisation

If you have a good think about it, the VCO input MOSFET, labelled Q_{in} in Figure 14, seems like it may be accessed and characterised if the external resistors are not present (see Figure 15 for the type of plot we might be interested in). Doing so may provide an insight into the VCO's operation (and indeed it might permit a realistic model of the VCO and its limitations). It's rather a complicated procedure, needs program control of two input voltages (V_G and V_{DS}) and a way to measure current (I_{DS}) reliably, and depends on there being no significantly impacting unknowns in the circuitry inside the 4046 chip (the figure is merely 'plausible'!). It has also not been assigned as a possible task in E1/E2 before, though I have tried it and obtained a reasonable (though slightly unusual) characteristic. You have been warned!

7.5 Other Relevant Applications

Occasionally something entirely different is attempted in E1/E2. At this stage of your career you should be learning the art of independent investigation, so if you have a smart idea which is relevant to the phase-locked loop, consider having a go. Having said that, maybe it's pragmatic to run it by your Head of Class, who will be marking your write-up and may not be quite so enthused!

References

Arduino AG. (n.d.). *Arduino Reference Website*. Retrieved from https://www.arduino.cc/en/

Arduino AG. (n.d.). *Arduino UNO Specification*. Retrieved from https://www.arduino.cc/en/Main/arduinoBoardUno/

Atmel Corporation. (n.d.). *ATmega328P Specifications (.pdf).* Retrieved from http://www.atmel.com/Images/Atmel-42735-8-bit-AVR-Microcontroller-ATmega328-328P_Datasheet.pdf

Cleaver, J. R., & Weaver, D. J. (n.d.). Microelectronics Research Centre, Cavendish Laboratory, unpublished.

National Instruments. (n.d.). *NI Website*. Retrieved from http://www.ni.com/

NXP. (n.d.). *Manufacturer's Data Sheets*. Retrieved from http://www.nxp.com/

pico Technology. (n.d.). *PicoScope 2000 Series website*. Retrieved from https://www.picotech.com/oscilloscope/2000/picoscope-2000-overview

Selected Further Reading

A. B. Grebene, 'Bipolar and MOS analog integrated circuit design', Wiley (2003)

P. Horowitz and W. Hill, 'The art of electronics', Cambridge University Press (1989)

S. M. Sze, 'Semiconductor devices: physics and technology', Wiley (2002)

Appendices

A1 Experimental Apparatus

A1.1 The Prototyping Board

Circuits are assembled on an RS 489-100 prototyping board, Figure A1(a). Power is available from a +5 V/0 V paired terminal and a variable-voltage three-terminal supply; the two 0 V terminals are independent and float relative to each other unless suitably connected. Circuits are constructed by pushing components and wiring into the 'breadboard', which has sockets which are internally connected in horizontal banks of five, as shown in Figure A1(b). Power is provided to the breadboard by wiring the output terminals to the four supply rails, which run vertically as shown. Figure A1(c) shows the appropriate way to mount an integrated circuit across the central division to avoid short-circuiting opposite pins. Take care to identify which end is which; there is a notch (or occasionally a dot) at one end of every integrated circuit package to define orientation when using the pinout diagrams in Appendix A3.

Wear and tear on the breadboards means that occasionally there may be defects such as loose connections; similarly, integrated circuits do occasionally 'blow'. Bear these in mind but remember that circuit failure is *almost always* the result of a wiring mistake.

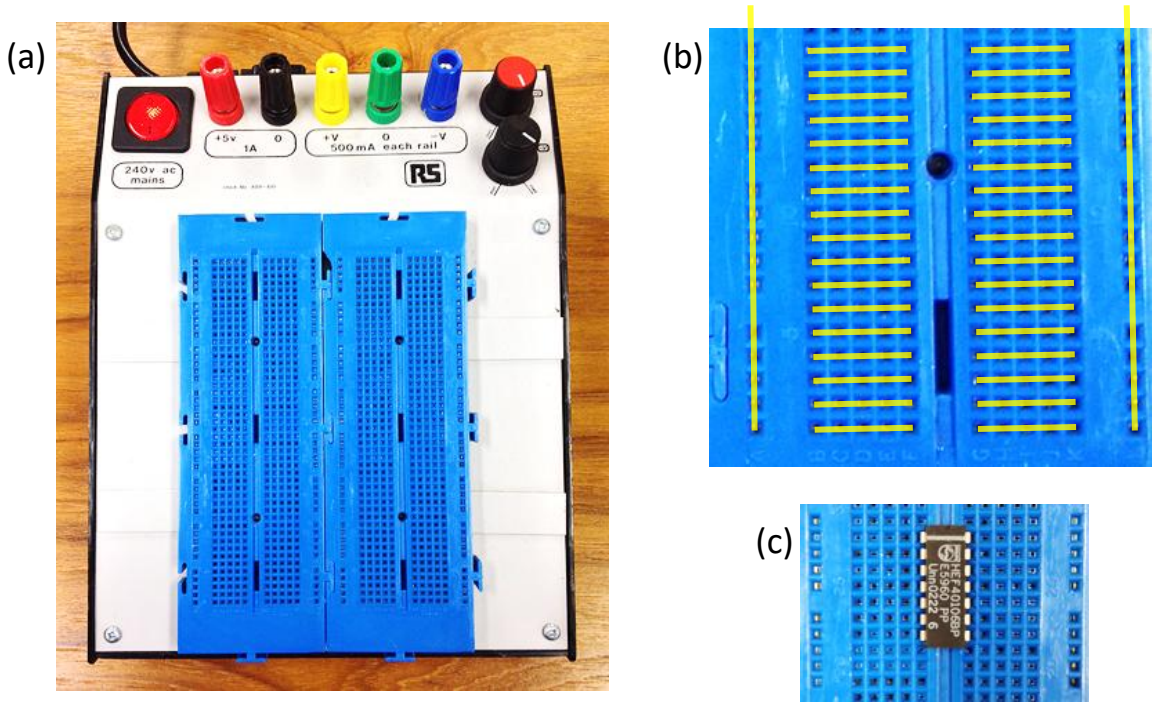


Figure A1 - RS prototyping board; (a) overview, (b) electrical linkage of pinholes with power lines on left and right, (c) correct positioning of integrated circuit across central gap.

A1.2 Using your own Laptop

Setting up your own laptop such that Python communicates successfully with your PicoScope can be frustrating. Again, **you are advised to use the lab laptops**, but I imagine there's a fair chance that you'll ignore me. In that case, here are some tips for Windows machines.

- Arduino IDE and Picoscope 7 are straightforward installations, and I advise using a programming environment more sophisticated than MS Notepad. MS Virtual Studio Code is installed on the lab machines, but hopefully your own favourite will work.
- Next, install the appropriate PicoScope SDK (Section 2.4.1) – again, not a problem.
- Python does not always co-operate, and using the version from the Windows Store may give problems. If whatever version you have works, go with it. If not, download and install the latest Python from https://www.python.org/ - there is a good chance that you may need to uninstall current instances of Python first. Without extensive testing, I believe ticking "Add python.exe to PATH" on the first screen is necessary, and I also tick the admin privileges box (which probably isn't).
- There are test programs in the "Python Code" subfolder of the "E1 E2 PLL files master" folder. At this point, `hello_world.py` should work (if not, try another Python installation strategy), and `hello_modules.py` should run but throw up errors – most likely the "matplotlib" module will not be found. Later you'll get a similar error about the "picosdk" module. In both cases, close your programming environment (important!) and install the modules from the Windows CMD (or PowerShell or other) terminal with whichever of these commands is appropriate:

```
python -m pip install -U matplotlib
python -m pip install -U picosdk
```

  The "picosdk" install won't work if you haven't already installed the PicoSDK (Section 2.4.1)!
- At this point , running `hello_modules.py` should give a friendly greeting, `hello_picoscope.py` should do nice things (if the PicoScope is correctly plugged in), and `control.py` should do something resembling a test measurement (but the AWG channel of the PicoScope should be connected to the "A" input channel or it won't trigger).
- If it still doesn't work… good luck; maybe try the lab laptops after all.

Anyone ~~silly~~ intrepid enough to try MacOS, Linux or other – I'd love to know about how it went. But really, time is precious, so maybe don't!

A1.3 Programming the UNO – Advanced Notes

Brief Comments on Syntax

As described in the reference material, when programming the Arduino, variables and constants must be declared in a specific way (and usually at the start of the sketch). There are other varieties (see online reference), but those listed in Figure A2 should be more than adequate. You will certainly have problems if your variables are defined or used wrongly, and variables MUST be defined before being used for the first time.

```
// note - two forward slashes instructs the Arduino to ignore the rest of
// the line - 'commenting out' user notes.
// note also - this is NOT a useful script!


// logic
boolean myboolean=true;      // true or false


// numeric
int myint=10000;      // 16-bit integer (between -32,768 and +32,768)
long mylong=10000000;      // 32-bit integer (+/-2,147,483,647)
float myfloat=2.67816E21;      // 32-bit floating-point number - 6 to 7
                               // digits precision


//strings & characters
String mystring3="Testing";      // possibly the most useful way of
// declaring a string, but the examples below may be useful in some
// circumstances.  Note the capital 'S'!
char mychar1='A';      // single character
char mychar2=65;      // equivalent, using ASCII value ('A'=65)
char mystring1[7]="Testing";  // mystring1[0] is "T", mystring1[4] is "i"
char mystring2[]="Testing";  // equivalent, but the UNO works out
// the size of the string for you
// note - declare characters in single quotes and strings in double
// quotes.
```

Figure A2 - Some examples of variable declaration for Arduino.

Simple Pinout Examples

Figure A3 shows a simple Arduino sketch to flash the LED near pin 13 on the UNO board. Make sure you understand what each line is doing. Copy the sketch into the Arduino IDE and execute it (note - make sure the IDE is communicating with the UNO ('Tools'→'Port'). Modify it to produce different behaviours.

As well as observing the flashing LED, measure the response on your PicoScope, as measured at pin 13 (remembering about appropriate grounding of your probe). Try other outputs - which pins should you be using?

Now try programming one of the pulse-width modulation (PWM) pins. The command which allows you to write to a PWM pin is `analogWrite(pin,n);`, where '`pin`' is the pin number and '`n`' is an integer variable between 0 and 255. Observe the output on your PicoScope and see whether the mean output level is as you would expect.

```
int pin=13;     // declares the variable 'pin' as an integer and
                // assigns its value

void setup() {      // this loop is performed just once
  pinMode(pin,OUTPUT);     // pin 13 is set up to act as a voltage output
}

void loop() {      // this loop is repeated indefinitely
  digitalWrite(pin,HIGH);    // set pin 13 'HIGH', i.e. 5 V
  delay(1000);      // wait 1000 ms
  digitalWrite(pin,LOW);     // set pin 13 'LOW', i.e. 0 V
  delay(1000);      // wait 1000 ms
}
```

Figure A3 - Arduino LED-flashing program.

Serial Communication

Whilst executing sketches, the UNO is able to communicate with the outside world in real time, and its outputs may therefore be controlled during an experiment. This is accomplished via a serial link. Figure A4 is an example of a very simple sketch which sends and receives strings using this link. Copy and paste it into the Arduino IDE, upload the sketch to the Arduino, and open the IDE's Serial Monitor window ('Tools'→'Serial Monitor'). Type some text into the Serial Monitor. Hopefully you will receive the response you expected. Incidentally, 9600 baud is a rather old standard data rate; we could choose a faster rate, but there would be little point.

You can probably imagine the usefulness of manipulating numbers rather than strings in a physics experiment. Have a go at sending, manipulating and returning numeric data through the serial link, but don't at this stage create anything too sophisticated. The Arduino IDE has many functions for achieving this; feel free (but certainly not obliged) to browse the Arduino reference. You will probably find it useful to use `Serial.parseFloat();` or `Serial.parseInt();` in place of `Serial.readString();`, to read the Serial stream in the appropriate format.

Use what you've learned to make a (very simple) sketch which controls one or more data pin voltages based on what you type into the serial input.

```
String message="";      // declare any string variables used later
String reply="";

void setup() {
  Serial.begin(9600);      // Initialises the serial link at
                           // 9600 bits/sec (9600 baud)
}

void loop() {
  if(Serial.available()){      // Checks whether there are any characters
                               // in the serial input buffer
    message=Serial.readString();      // Reads the characters into a
                                      // string variable
    reply=String("You typed " + message);      // Creates a new string by
                              // concatenating a string and a string variable
    Serial.println(reply);      // Sends the new string back via the
                                // serial link
  }
}
```

Figure A4 - Serial communication program.

Speeding Things Up - Arduino UNO Pulse Generator

Your experiment is mainly concerned with the investigation of a digital integrated circuit used for tracking a digital signal, so it is potentially useful to be able to use the UNO to generate a reasonably fast digital, pulsed signal. For now, leave aside the serial communication and write a sketch which will do that, based on the commands `digitalWrite();` and `delayMicroseconds();`, the latter's purpose being reasonably obvious, allowing you to select the frequency programmatically. What can you say about the results you obtain? Any drawbacks or imperfections?

In practice, we need to cheat slightly to get a useful result and do away with `digitalWrite();`, which is not really designed for this purpose, in favour of some more *direct* coding.

The UNO has three 'ports' (don't worry what this means), of which we will only concern ourselves with port B, which controls pins 8 to 13. All of the pins in this section may be set up to be outputs or inputs, and the register '**DDRB**' controls this assignment. The bits in **DDRB**, reading from high to low, represent pins 13 to 8, and the command `DDRB=B010000;` (equivalently, you could use decimal, *i.e.* `DDRB=16;`) sets pin 12 to output mode and the rest to input (the latter 'B' denotes a binary number). To set pin 12 to **HIGH** (5 V), issue the command `PORTB=B010000;` (or decimal `PORTB=16;`). Reasonably obviously, set the relevant bit to zero in **PORTB** to set pin 12 **LOW**. Always set **DDRB** in the setup loop (it needs setting only once) and include with it the very useful `noInterrupts();`, which disables the 'interrupt' - the UNO doesn't wander off to check on other processes (try with and without and see what difference it makes). Note the principal features of your results, advantages and drawbacks, and consider how suited the UNO is for acting as a digital pulse generator.

At the risk of repeating myself, if you do not use `noInterrupts ();`, you are likely to get limitingly terrible jitter (fluctuations in pulse width) when using the UNO for anything except very slow operation; I would expect you not to take your "best" data without implementing it; some imaginative programming may be needed if the signal frequency is to vary during a measurement. For information, Interrupt may be re-enabled by `interrupts ();`.

You now know how to program the UNO.

A2 The Phase-Locked Loop as a Control System

A2.1 Linear, Small-Signal Analysis for Stability

A primary characteristic of any feedback system is its stability: that is, whether it provides constant output (for the phase-locked loop, a constant-frequency output) when the input conditions are held constant, or whether its output oscillates (here, the output frequency oscillates, so that the output signal is frequency-modulated).

We can consider the requirements for stable operation of the phase-locked loop by determining its response to small perturbations, whilst ignoring the process by which the circuit initially locks on the incoming signal. The linear, small-signal analysis of the system stability treats variations about the condition when the input frequency is equal to the free-running frequency of the oscillator. It is assumed that the comparator output voltage is linearly dependent on the phase difference, and that this voltage does not vary significantly over any one cycle of the voltage- controlled oscillator.

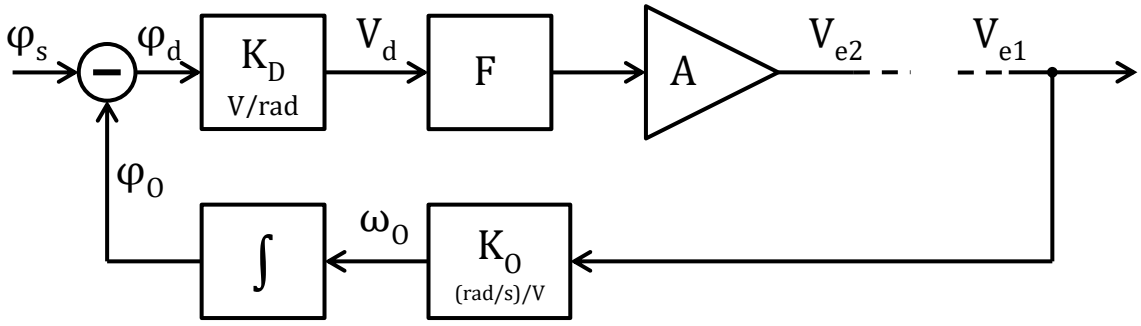


Figure A5 - Transfer function terms - circuit with feedback loop opened at Ve.

The feedback circuit is broken up into a sequence of 'transfer function' terms as shown in Figure A5. A standard way of assessing the stability of a feedback circuit is to consider its open-loop characteristics, and to predict from them what the characteristics would be with the loop closed. This is considered in more detail in 0, but the principle can be illustrated simply: suppose that the circuit is split at the output of the error amplifier, so that the amplifier output is separated from the input to the voltage-controlled oscillator. Suppose that a variable-frequency alternating signal is injected at the point at which the circuit is split (as the input to the voltage-controlled oscillator), and that for some injected-signal frequency the signal that returns to the other side of the split is in phase with the injected signal. If the magnitude and phase of the returning signal are the same as those of the injected signal, then the split could be reunited and the injected test signal removed, and nothing would change - steady oscillations would persist. If the returning signal was in phase, but larger, then the signal would oscillate but its amplitude would grow exponentially; if smaller, its amplitude would decay exponentially. Thus, we split the circuit at the error signal V_e to inject a signal V_{e1} into the voltage-controlled oscillator, and to investigate the signal V_{e2} (whilst keeping the input signal ω_s ($=d\phi_s/dt$) constant).

Assume that	$V_{e1} = V_{e10} + \Delta V_{e1}$
output frequency	$\omega_O = K_O (V_{e10} + \Delta V_{e1})$
output phase	$d\phi_O/dt = K_O (V_{e10} + \Delta V_{e1})$
but	$V_{e2} = K_D A F (\phi_s - \phi_O)$

with V_{e2} the sum of static and time-varying terms

$$V_{e2} = V_{e20} + \Delta V_{e2}$$

$$d/dt(V_{e2}) = K_D A F (d\phi_s/dt - d\phi_O/dt)$$

$$= K_D A F (d\phi_s/dt - K_O V_{e10} - K_O \Delta V_{e1})$$

Assuming the constant input signal and the unperturbed oscillator signals to be in phase, so that

$$\omega_s = \frac{d\varphi_s}{dt} = K_O V_{e10}$$

then with complex sinusoidal variations at the modulation frequency ω_m,

$$\Delta V_{e1} = \Delta V_{e10} e^{j\omega_m t}$$

and $$\Delta V_{e2} = \Delta V_{e20} e^{j\omega_m t}, \quad \Delta V_{e1}, \Delta V_{e2} \text{ complex}$$

then $$j\omega_m \Delta V_{e20} = -K_D A F K_O \Delta V_{e10}$$

In the basic case that the amplifier has flat response and negligible phase shift, and there is no filter,

$$\frac{\Delta V_{e20}}{\Delta V_{e10}} = \left(\frac{-1}{j\omega}\right) K_O K_D A$$

The phase-shift at the modulation frequency ω_m is always π/2, giving the Bode diagram in Figure A6. Therefore, the system is inherently stable, since it would be necessary for the phase-shift to reach π for the system to oscillate (there is a further phase-shift of π due to inversion at the amplifier, since the system is designed to provide negative feedback).

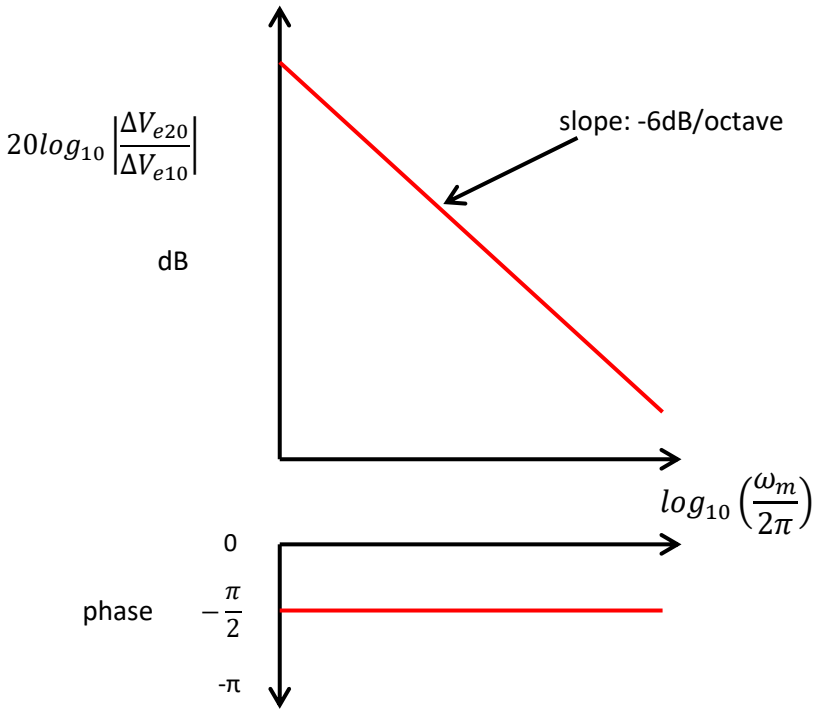


Figure A6 - Bode diagram for basic system without filter.

A2.2 Effects of Filter Circuits on Stability

The idealised basic system is inherently stable but is very limited in its applications. Real comparator signals vary over a cycle of the signal frequency, and this variation may need to be smoothed out. It may be required that the voltage-controlled oscillator provides a steady output even if the signal is subject to noise or missing cycles (in clock signal regeneration, for instance) or if the frequency of the signal applied to the comparator is different from the frequency of the oscillator (as is a frequency multiplier).

In such cases, the obvious approach is to interpose an R-C integrator as the smoothing circuit, to filter the comparator signal before applying it to the voltage-controlled oscillator.

However, the effect of the integrator is to introduce further phase shift at the modulation frequency, so that as ω_m becomes large the added phase shift around the loop approaches π (and can exceed π if the other phase-shifts are present). Thus, there is potential for ringing or even oscillation.

In summary, for the R-C integrator of Figure 7,

$$V_2/V_1 = (1+j\omega CR)^{-1}$$

To give Bode diagrams for the integrator alone and for the combination of the integrator with the remainder of the phase-locked loop as in Figure A7.

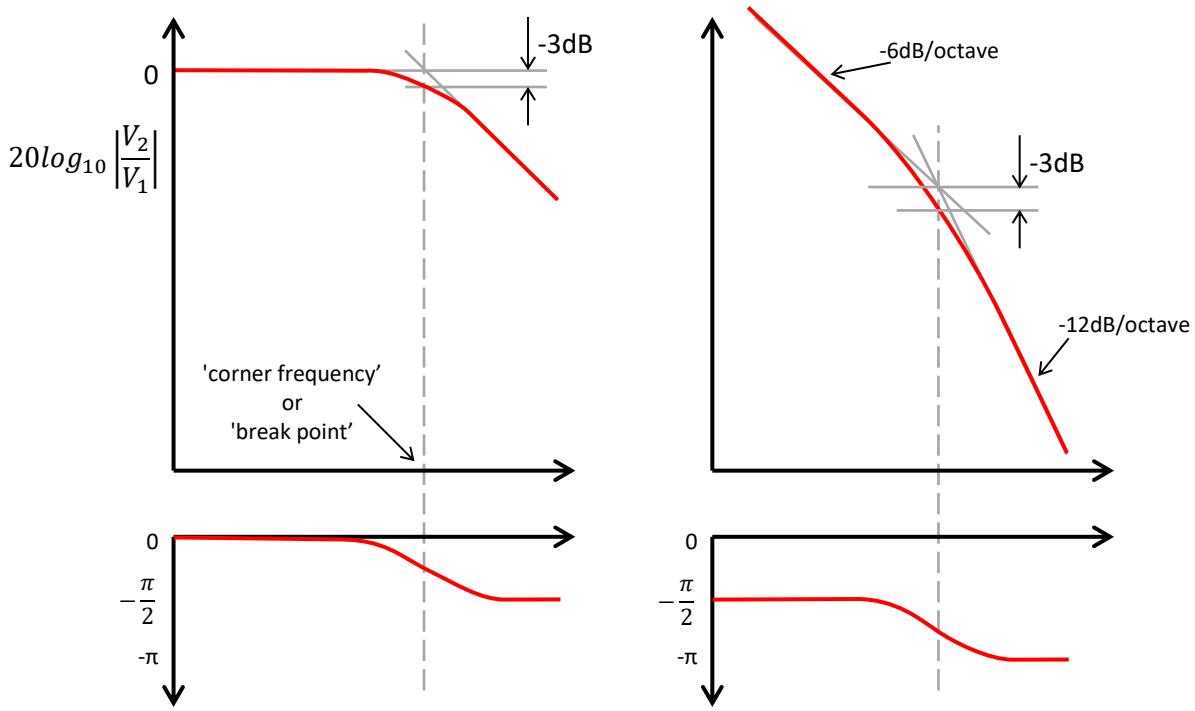


Figure A7 - Bode diagrams for the integrator alone, and for a complete system with an integrator.

A variant on the basic integrator is the lead-lag circuit of Figure 8, in which the phase-shift is zero at both low and high frequencies, and in which the phase-shift never reaches π/2.

In this case $$\frac{V_2}{V_1} = \frac{1+j\omega C_2 R_2}{1+j\omega C_2 (R_1+R_2)}$$

or $$\frac{V_2}{V_1} = \left(\frac{1+\omega^2 C_2^2 R_2^2}{1+\omega^2 C_2^2 (R_1+R_2)^2}\right)^{\frac{1}{2}} \left(\frac{\exp j\varphi_1}{\exp j\varphi_2}\right)$$

total phase-shift is $$tan^{-1}(\omega C_2 R_2) - tan^{-1}(\omega C_2 (R_1 + R_2))$$

If the maximum phase-shift is evaluated, it will be seen to depend only on the resistor ratio, and so this ratio can be chosen to ensure an adequate phase margin for a complete system (irrespective of value of capacitor, which sets the frequency of maximum phase shift). The relevant Bode diagrams are shown in Figure A8.

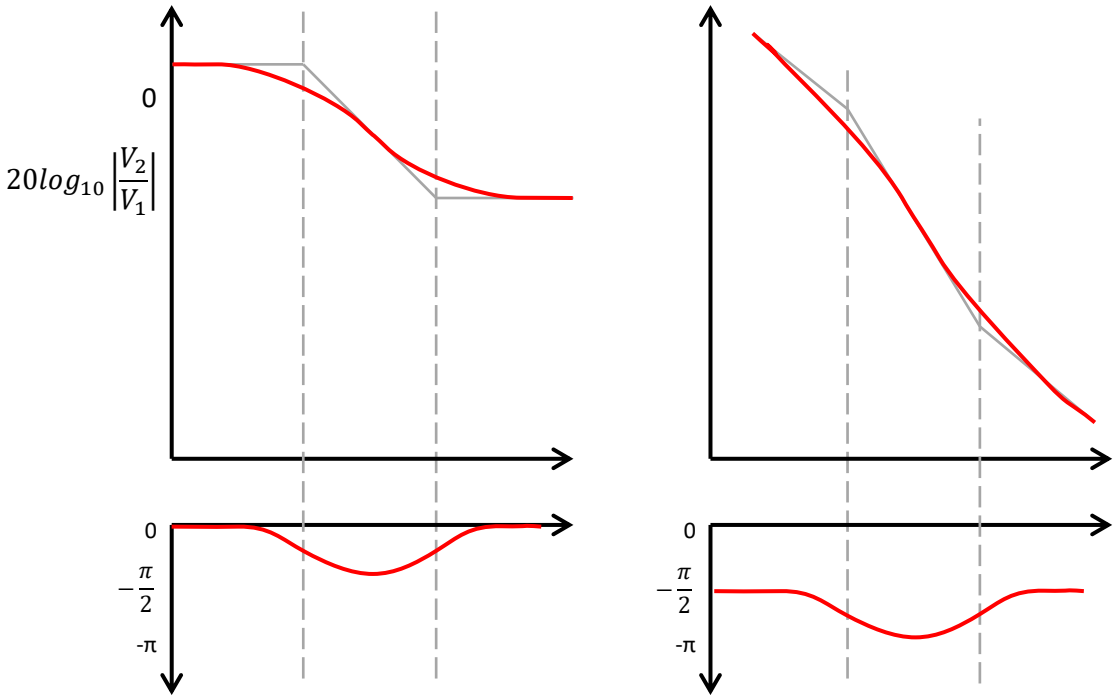


Figure A8 - Bode diagrams for the lead-lag circuit alone, and for a complete system.

A2.3 Gain, Margin, Phase Margin and Acceptable System Performance

The necessary condition for any feedback system, that it should not oscillate and should eventually settle to a static condition, is not sufficient for acceptable performance since it implies neither that the response to sinusoidal inputs will be satisfactory at all frequencies, nor that the transient response will be satisfactory. Examples of unsatisfactory behaviour are shown in Figure A9.

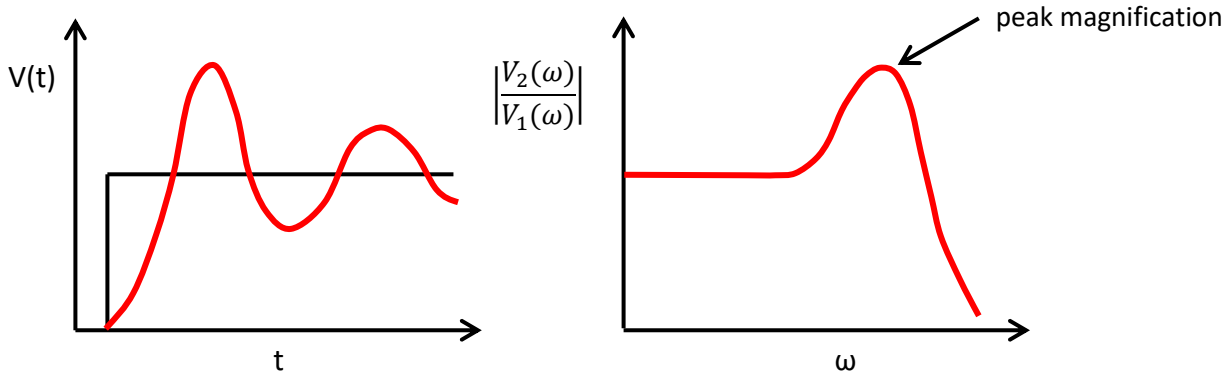


Figure A9 - Unsatisfactory response, shown in the lime domain and in the frequency domain.

Consider a feedback system as in Figure A10(a), for which the open-loop characteristic V_2/ε is shown in Figure A10(b); this polar plot of the relationship between the 'error' signal and the output is known as a Nyquist plot. Then

$$V_1/\varepsilon = V_2/\varepsilon + 1$$

and the ratio between the input signal and the error signal on closed-loop is given by the open- loop plot with its origin shifted to (-1,0); oscillation at steady amplitude corresponds to the open-loop locus passing

though (-1,0), since then no input is required to sustain the closed-loop error and output signals, Lines of constant closed-loop magnification are plotted in Figure A10(c).

One criterion that can be applied is that (irrespective of phase shift) the closed-loop magnification should never be excessive, perhaps twice the low-frequency magnification. Then

$$2\,|V_1/\varepsilon| = |V_2/\varepsilon|$$

This defines a region on the Nyquist plot, bounded by a constant-magnification locus, which the locus of the open-loop gain must not enter. Rather than using the constant-magnification locus directly, it is convenient to use criteria based on two specific points on the curve.

For **gain margin**: when the phase shift is π, the locus must not pass to the left of the marked point on the negative real axis. The amount by which the gain must fall short of unity (0 dB) is known as the gain margin and is generally expressed in dB.

For **phase margin**: when the modulus of the open-loop gain is unity, the phase-shift must be less than π, by an amount known as the phase margin.

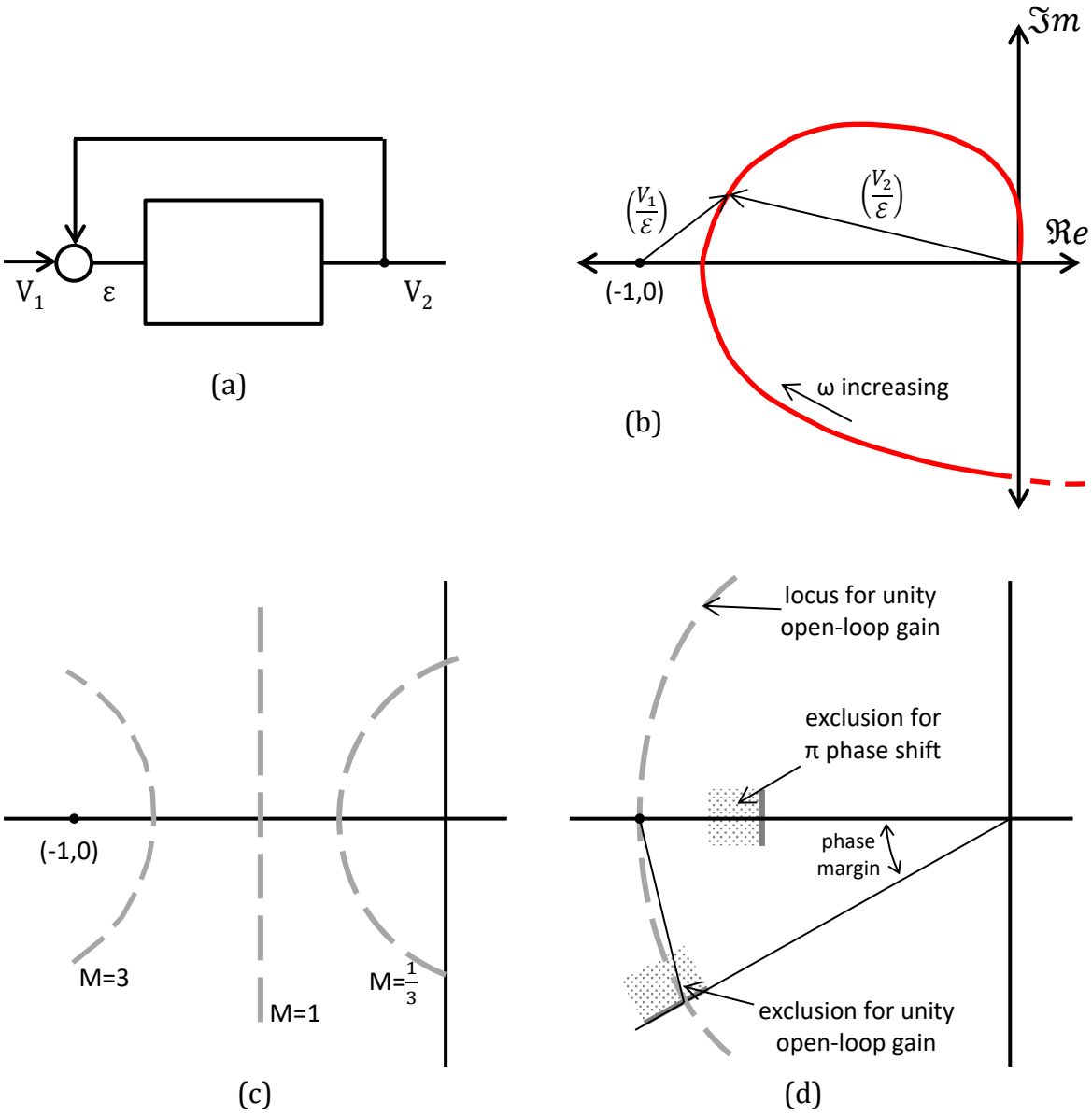


Figure A10 - Open-loop and closed-loop characteristics - the Nyquist plot.

A2.4 Bode Diagrams

The Bode diagram is a plot of the log of the modulus of the open-loop gain (expressed in dB) against the log of the frequency. It is supplemented by a plot of the phase shift against the log of the frequency.

The log-log form of the plot facilitates representation of system characteristics over a wide frequency range and enables much of the plot to be approximated by straight lines. For instance, consider the simple R-C integrator, Figure 7, for which

$$(V_2/V_1) = (1+j\omega CR)^{-1}$$

In the low-frequency limit: $|V_2/V_1| \rightarrow 1$

In the high-frequency limit: $|V_2/V_1| \rightarrow 1/\omega CR$

The two straight lines intersect at $\omega = 1/CR$, where the value of $|V_2/V_1|$ is $1/\sqrt{2}$, -3 dB, and so the correction can be made and the characteristics sketched in.

Because the Bode diagram plots gain and phase separately against frequency, it is particularly suitable for use with gain-margin and phase-margin criteria. For example, the frequency at which the phase shift is π can be identified, and at that frequency the gain must be -(gain margin) dB; alternatively, at the frequency at which the gain has fallen to unity the phase shift must be (π-phase margin).

A3 Integrated-Circuit Connections

All the circuits used are in dual-in-line plastic packages.

All the HEF4000 series CMOS logic circuits can be operated off a +5 V supply.

All figures in this Section are reproduced from the manufacturer's data sheets (NXP, n.d.).

A3.1 HEF4046B and 74HC4046 Phase-Locked Loops

The HEF4046B, on which this experiment is based, is currently unavailable from our suppliers. Please use the remaining HEF4046B chips, but if they run out, you will find 74HC4046, which are functionally the same apart from a third type of comparator (on pin 15, replacing the 'Zener' output on the old chip) which we will not be investigating here. Pinouts are shown in Figure A11.

NOTE: The 'inhibit' input must be grounded for the voltage-controlled oscillator to operate!

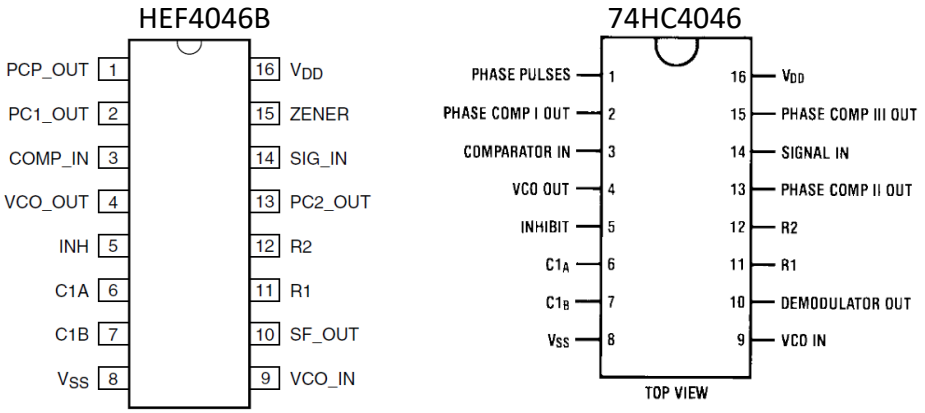
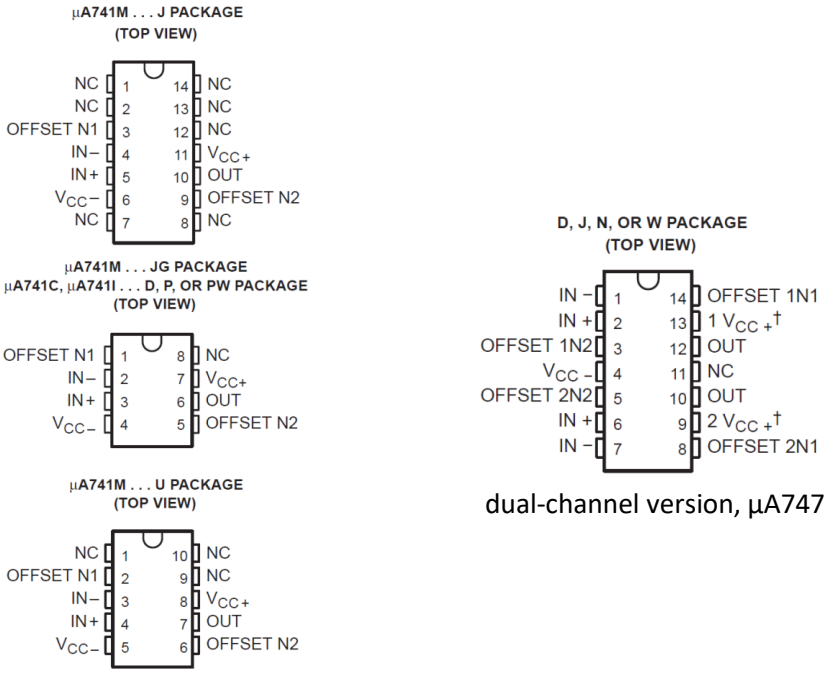


Figure A11 - The 4046 family of phase-locked loop integrated circuits.

A3.2 µA741 Operational Amplifier

This circuit (Figure A12) is operated with symmetrical power-supply rails, here ± 15 V. There are many versions of the 741, and many alternatives (often with better performance) exist.



dual-channel version, µA747

Figure A12 - µA741 family of operational amplifiers.

A3.3 TL071 Operational Amplifier

The TL071 (Figure A13) is a nice general-purpose op-amp with respectable higher-frequency performance. This may be the best choice available to you. Again, it runs from a ± 15 V supply.

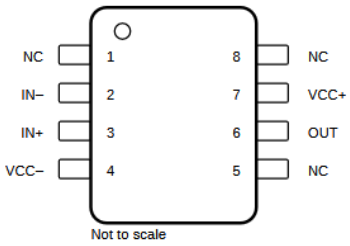


Figure A13 - TL071 operational amplifier.

A3.4 LM393A Dual Comparator

This circuit (Figure A14) can be operated with ±15-volt supplies, for which 'V^+' becomes +15 V and 'GND' becomes -15 V. In this case, the output switches between +15 V and -15 V.

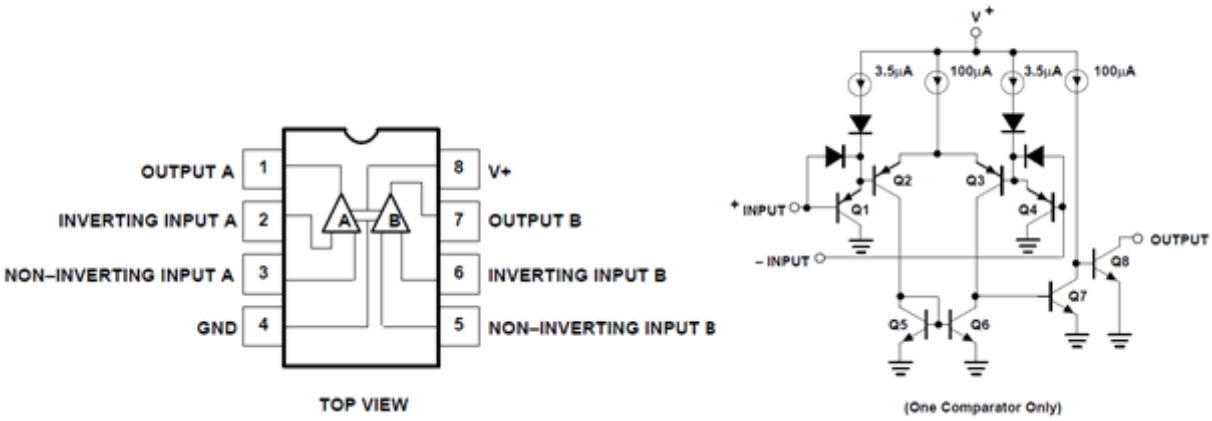


Figure A14 - LM393A Dual Comparator.

Note that the comparator has an 'open-collector' output, with the collector of the output transistor connected directly to the output pin. In this arrangement, if the output transistor is turned ON, it can pull the output line down to 'ground' to give the 'logic 0' level, but when it is OFF there is no internal means for pulling the output up to the 'logic 1' level; this must be done externally, by connecting the output via a load resistor to a positive supply rail. The load resistor should be chosen so that the transistor current does not exceed 10 mA.

A3.5 HEF4015B Dual 4-Bit Static Shift Register

The circuit outputs (Figure A15) consist of successively-delayed equivalents to the input. The 'master reset' input MR should be held low.

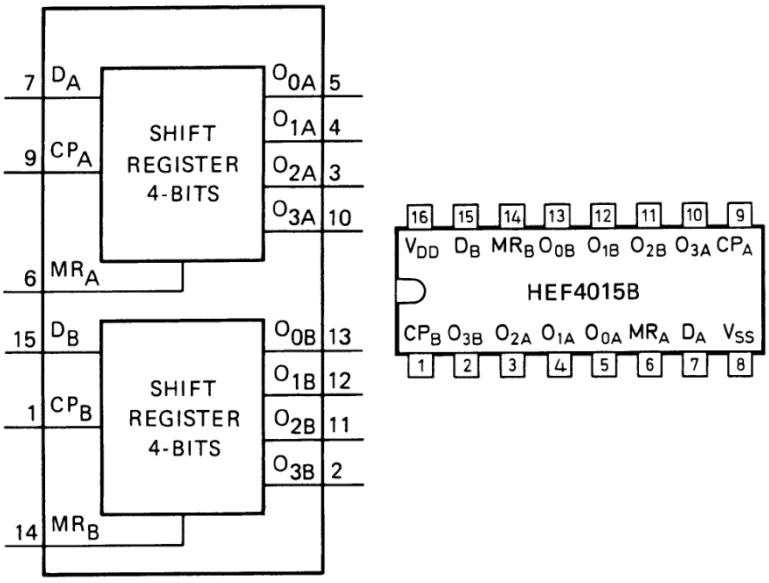


Figure A15 - HEF4015 shift register.

A3.6 HEF4020B and CD4020BE 14-Stage Binary Counters

The HEF4020B consists of a sequence of divide-by-two stages (Figure A16). The 'master reset' input MR (or 'RESET' on the CD4020B) should be held low. As with the HEF4046 phase-locked loop chip, the HEF4020 counter is no longer supplied, so you may need to use the (equivalent) CD4020BE (Figure A17) instead.

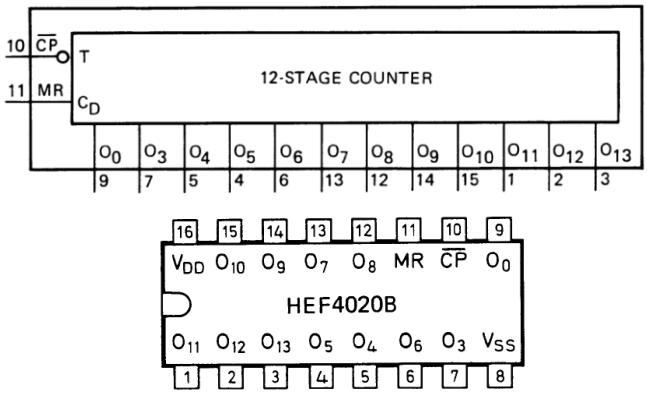


Figure A16 - HEF4020B binary counter.

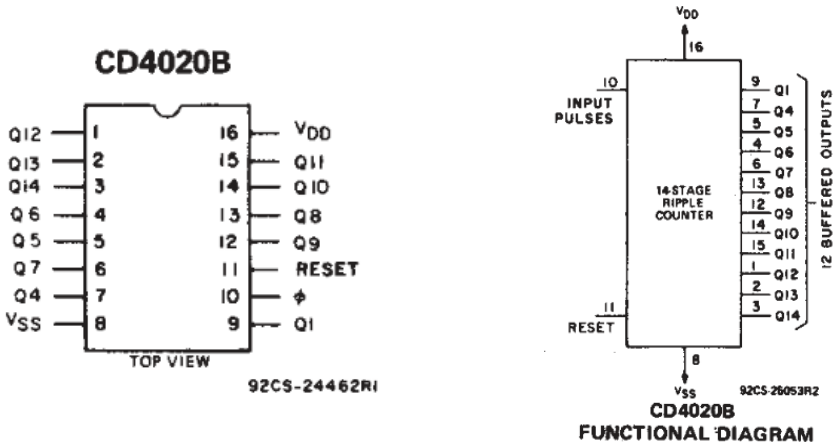


Figure A17 - CD4020B binary counter.

A3.7 HEF4081 B Quadruple 2-Input AND Gate

The HEF4081 B (Figure A18) is a seven-pin IC with four logic AND gates.

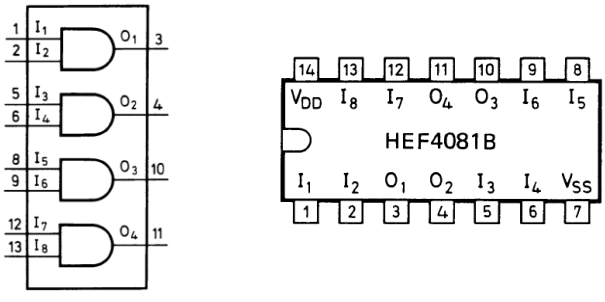


Figure A18 - HEF4081 B AND gate.

A4 Operational Amplifier Circuits

It is quite likely that you will not need to use an operational amplifier, or 'op-amp', to modify a signal, but it can be very useful in some circumstances, particularly when producing a 5 V logic-level pulse from the PicoScope's 2 V-limited arbitrary waveform generator (AWG) output. These common circuits are provided for your reference, and without expectation that you will make use of this section.

Figure A19 shows some commonly-used op-amp configurations. The buffer circuit has unity gain, but its high input impedance makes it useful for minimising loading problems; the buffer supplies a voltage the same as that at its input even when load circuits draw high currents. The other circuits should not need describing, and their properties are straightforward to calculate.

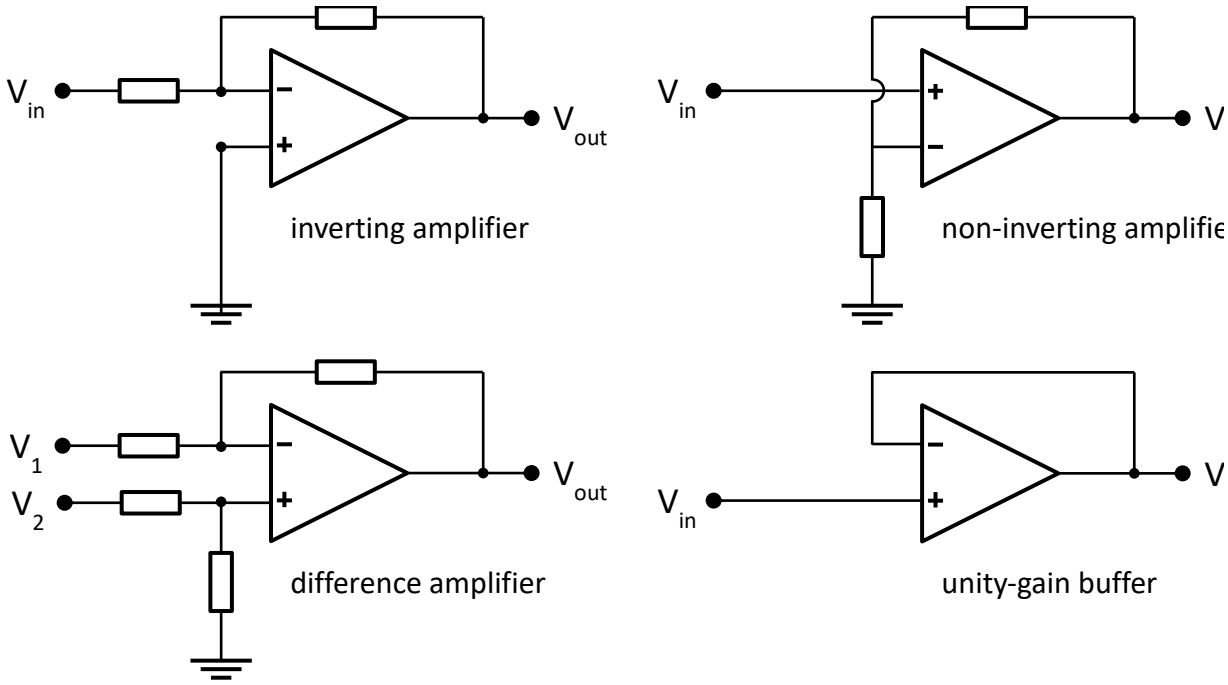


Figure A19 - Commonly-used configurations for operational amplifiers (supply voltages not shown).

A5 Physical Layout of the HEF4046B

In this section, some information is provided for understanding the physical construction of the 4046. It is included for interest only, having been part of an older version of this experiment. You are NOT expected to read or understand this section!

A5.1 Introduction

It is well worth acquiring an insight into how the HEF4046B chip works as well as what it does. Despite its great simplicity and large feature size compared with modern processors and memory chips, the HEF4046B still appears complex. However, once you have an idea what you're looking at, many features can be identified. The numbered input connections on the micrograph (Figure A20) can immediately be matched up with the corresponding chip package terminals (Figure 9), but some further knowledge of chip manufacture will be helpful.

A5.2 Integrated-Circuit Layout and Operation

The 4046 phase-locked loop integrated circuit is currently in production, despite being quite an old design (the original 4000-series CMOS circuits were first introduced in about 1980). In consequence, the individual elements of the integrated circuit (transistors, conductor tracks and suchlike) are quite large and can be seen readily under an optical microscope. For instance, the minimum conductor-track width in the circuit is about 5 µm (for comparison, modern high-performance mass-produced processor chips have feature sizes approaching 10 nm).

Figure A20 is an optical micrograph of the circuit chip, which is actually about 1000 µm by 1200 µm. If you're interested, it may be useful to relate some conclusions to this figure.

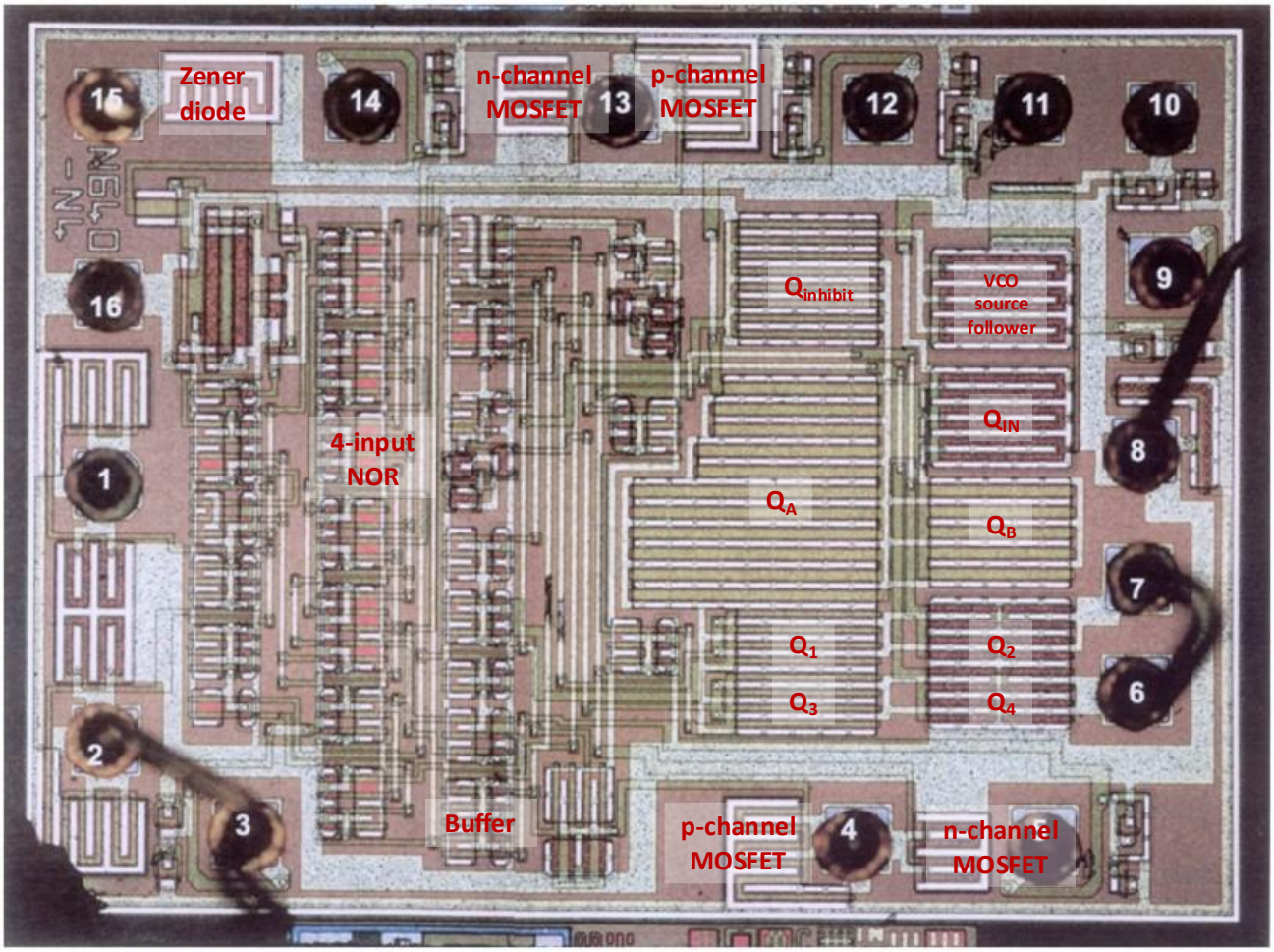


Figure A20 - HEF4046B chip annotated to show key areas, with pins numbered as in the pinout diagram (Figure 9). The logic circuits make up the bulk of the left-hand side. See elsewhere in this manual for description. Chip reverse-engineered by D. J. Weaver. (Cleaver & Weaver)

A5.3 Example 1: Four-Input NOR Gate

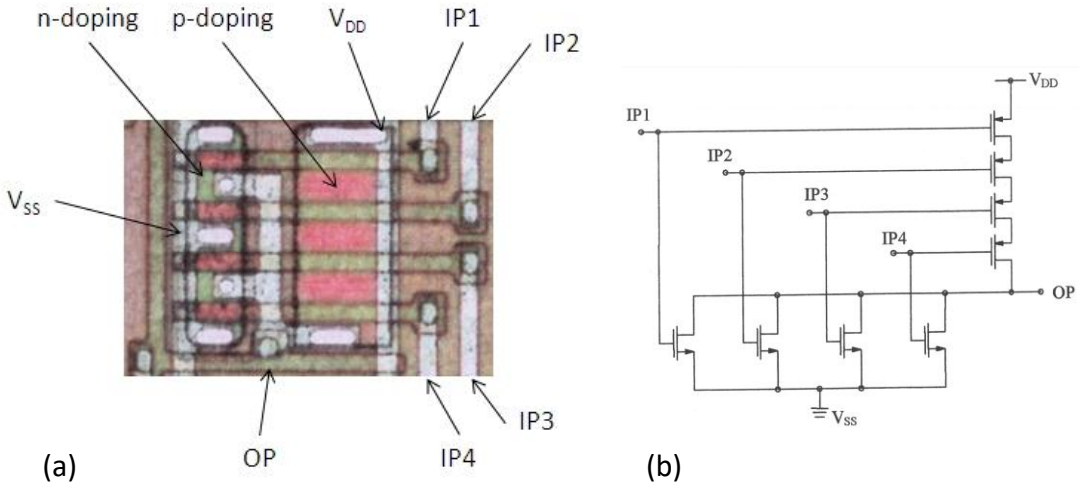


Figure A21 - (a) Micrograph of a four-Input NOR gate from a HEF4046B and (b) its equivalent circuit.

As an example of one of the basic units of the HEF4046B, Figure A21 shows a four-input NOR gate (found in Figure A20 at centre-left), along with its circuit equivalent. Some correspondence between the two figures can be picked out immediately - the vertically-aligned silver-coloured tracks on the right of Figure A21(a) are metal interconnects feeding in the four input voltages, IP1 to IP4. These link to greenish horizontally-oriented tracks which play a dual role - like the metal tracks, they are simple electrical conductors, but they also act as the gate electrodes for transistors. They are made of implantation-doped polycrystalline silicon.

It should now be possible to pick out two banks of four transistors in both parts of the figure; the large pinkish rectangle in the centre is an area of p-type silicon, and belongs to the bank of transistors running vertically in the schematic of Figure A21(b). Notice that the four transistors connect in series, having metal contacts to the outside world only at the ends of the bank. By contrast, the other set of transistors, which run horizontally in the schematic and vertically (at the left) in the micrograph, are connected at each intermediate point.

One remaining issue is isolation. The micrograph has numerous apparent intersections where conductors cross, for example where the vertical V_{DD} line runs over the IP1-4 gates. In fact, there is a layer of insulating material, probably SiO_2 or similar, between the silicon/polysilicon parts and the metal tracks, and the metal contacts the buried devices only through small holes etched in that insulator - you can see several of these holes in the micrograph, most obviously on the IP1-4 inputs. The insulator is probably grown by oxidising the chip in an oxygen furnace, or perhaps by one of several layer deposition techniques.

A5.4 Process Flow

A likely process flow, somewhat simplified, for the four-input NOR gate of Figure A21 is shown in Figure A22. Processes will be more complicated in the actual fabrication line, and in reality will differ significantly from this illustrative flow.

The process starts with a (probably lightly p-doped) silicon wafer (a), perhaps having a buried SiO_2 insulator layer (not shown) which helps with good electrical isolation and performance. The transistor areas are defined by masking, perhaps with SiN, and the wafer is given a prolonged oxidation in a furnace under oxygen flow before the mask is etched off, leaving the majority of the wafer's surface covered by highly insulating SiO_2 (b). A fresh layer of silicon (which will change from amorphous to poly-crystalline during later

thermal treatments) is deposited across the whole chip by sputtering then etched away where it is not needed, defining transistor gates and some other interconnect tracks (c). High-voltage implantation of dopant ions will make the poly-crystalline silicon highly conductive and is also used to define the transistors' drain and source contacts by doping the silicon (d). An insulating layer is formed across the wafer surface (e) by a further oxygen furnace step, or perhaps by spin-coating with a glass precursor and subsequent heating. Where contacts are needed to the buried structures, holes are etched in the insulator (f), before the final metallisation is performed (g), again by sputter-depositing metal across the whole wafer and etching away the unwanted material. A metal such as aluminium (saturated with a few percent silicon to prevent substrate dissolution) may be used in low-tech applications.

A key point to bear in mind is that the metal tracks only ever make electrical contact with the polysilicon tracks via the etched holes in the dielectric. The two layers cross each other in very many locations without making any electrical connection.

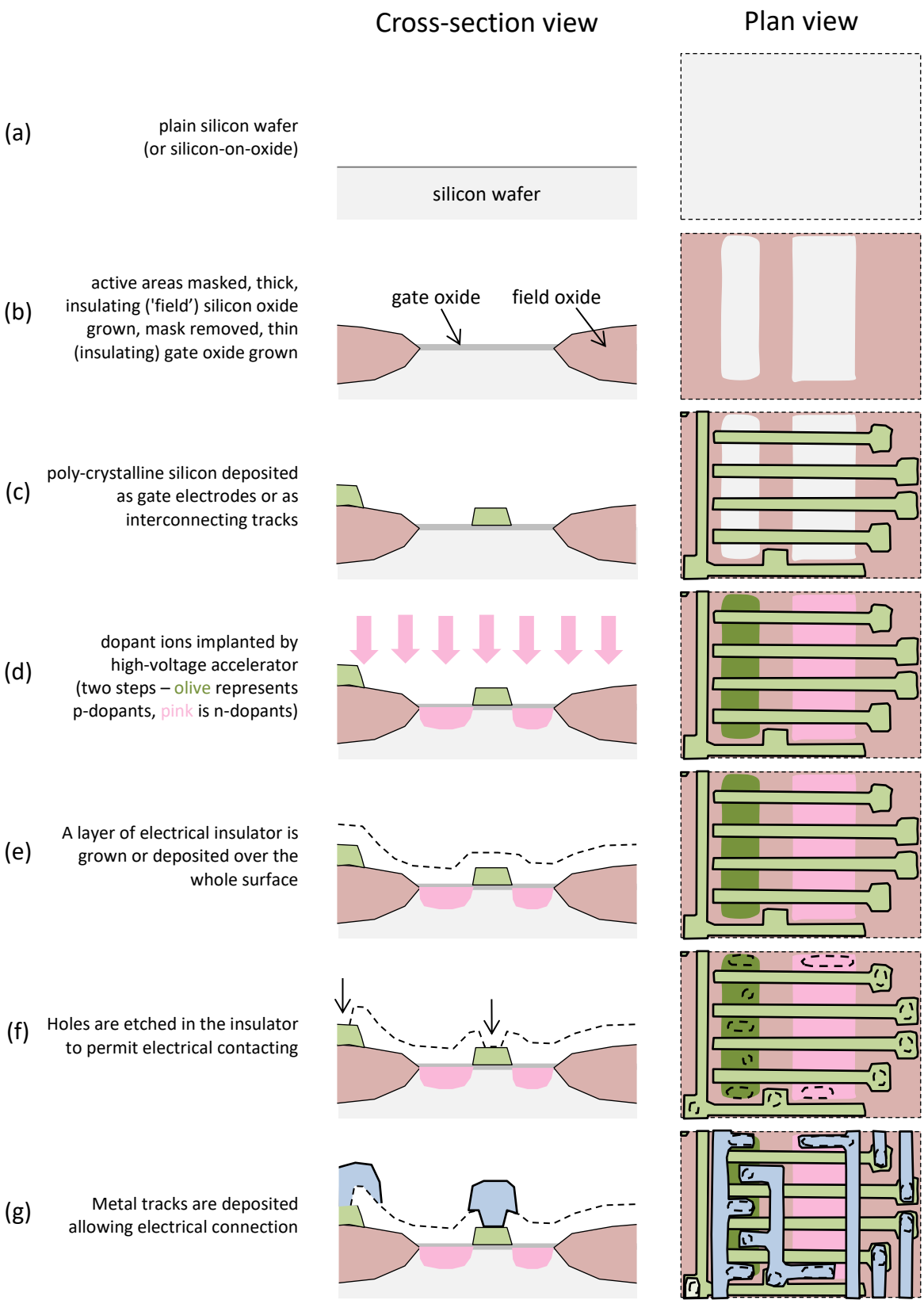


Figure A22 - Plausible simplified process flow for the HEF4046B chip, with the layout of the four-input NOR gate of Figure A21 as an example.

A5.5 Example 2: Large Channel-Width Transistor

The HEF4046B chip needs to control some fairly large currents (compared with those used for logic) for certain of its duties, and therefore it incorporates some high-current transistors. To accomplish a suitably high current flow, a design with a large channel width is incorporated. One such transistor is examined in Figure A23. By interdigitating the polysilicon gate (b) with metal drain/source contacts (d), the block is equivalent to a single transistor of width several times the physical width of the block. Note that the vertically-oriented metallic strip in (d) crosses the insulating dielectric and therefore is only in electrical contact with the 'source'-type rows of the device. Similarly, the metal strips of the 'drain' contacts either side of this spine are joined beneath it via the doped region in the substrate. Other transistors on the HEF4046B have different configurations, but similar principles in terms of interconnection.

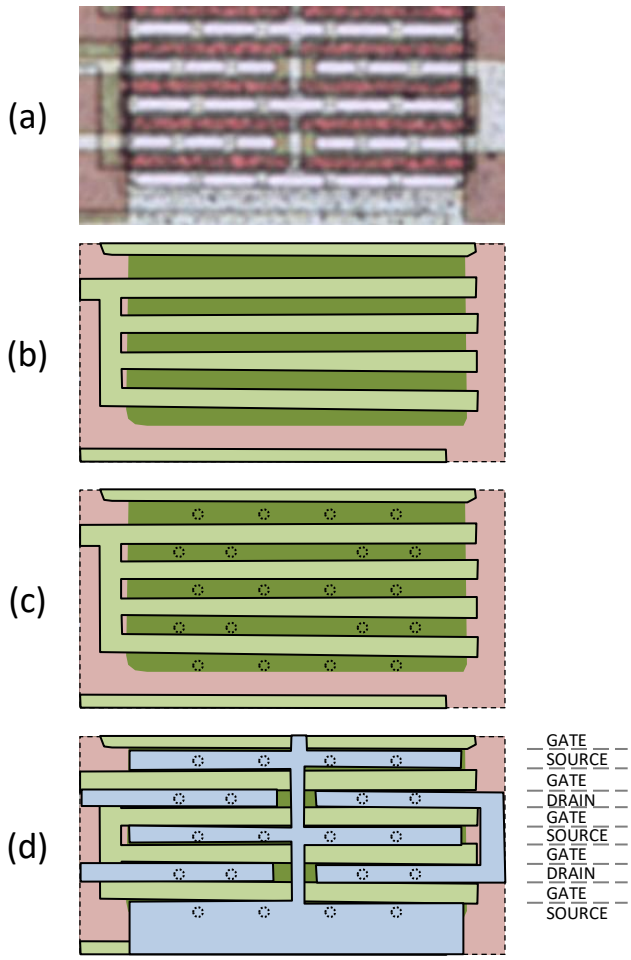


Figure A23 - Large-area transistor from HEF4046B chip showing (a) micrograph extract, (b) position of gates over the active region, (c) electrical access holes etched through the insulating dielectric, (d) metallisation, indicating the purpose of each horizontal strip.